

BUDAPESTI MŰSZAKI ÉS GAZDASÁGTUDOMÁNYI EGYETEM
VILLAMOSMÉRNÖKI ÉS INFORMATIKAI KAR
MESTERSÉGES INTELLIGENCIA ÉS RENDSZERTERVEZÉS TANSZÉK

AMD Vitis HLS

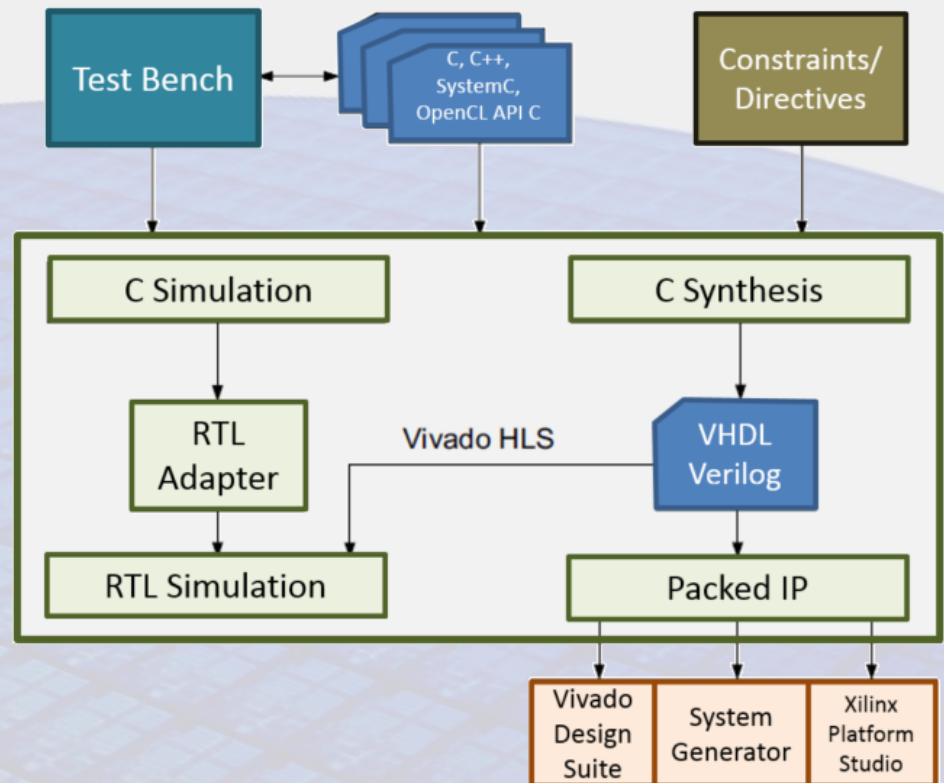
Szántó Péter
FPGA labor

Vitis HLS

- **Magas szintű szintézis: C++ leírásból hardver**
 - A C szekvenciális
 - A hardver párhuzamos
- **NEM meglévő C kódból egy klikkes hardver generálására való**
 - Számos konstrukció nem támogatott
 - Sokszor kell a HLS saját típusait és könyvtárait használni
- **A C kódba elhelyezett direktívákkal (pragma-k) adjuk meg, hogy milyen HW-t szeretnénk**

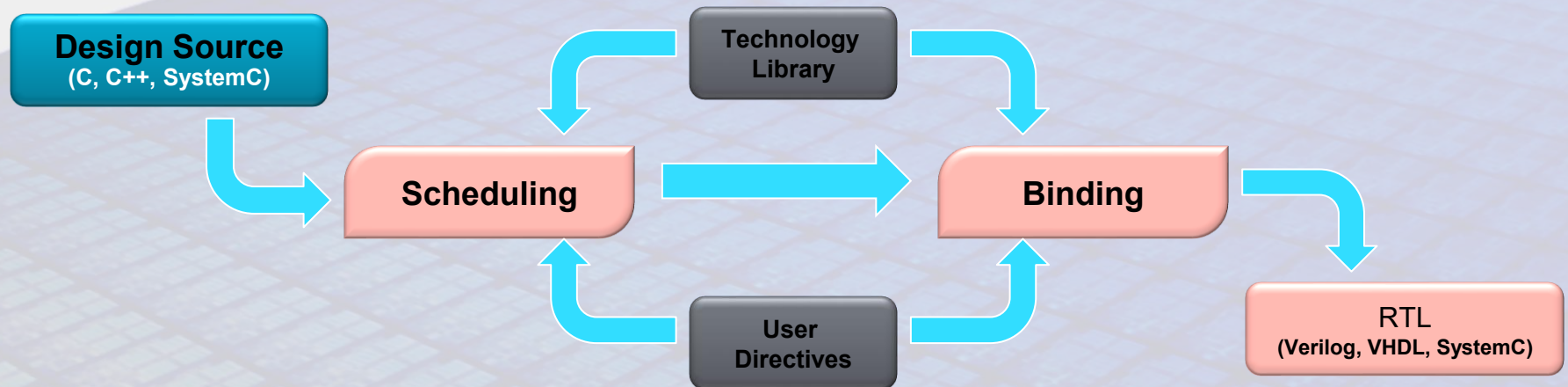
Miért?

- C-ben gyorsabb ugyanazt a funkciót leírni
- Egyetlen kódból többféle HW
 - A kód funkcionális változtatása NÉLKÜL
 - Direktíva változtatással
- Ellenőrzés
 - C kód ellenőrzése C testbench segítségével
 - Ugyanaz a C testbench használható a HDL ellenőrzéséhez



Működés

- Ütemezés és erőforrás allokáció
- Ütemezés
 - Adott műveletet melyik órajelben lehet elvégezni
- Erőforrás allokáció: milyen HW elemet használ
 - FPGA típus, jelterjedési idő, direktívák



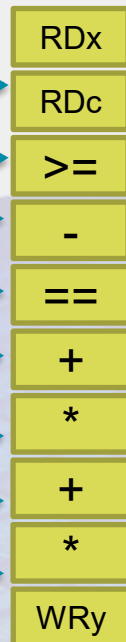
Vezérlés és adatutak

C kód

```
void fir (
  data_t *y,
  coef_t c[4],
  data_t x
){
  static data_t shift_reg[4];
  acc_t acc;
  int i;

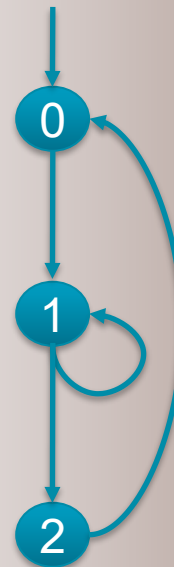
  acc=0;
  loop: for (i=3;i>=0;i--) {
    if (i==0) {
      acc+=x*c[0];
      shift_reg[0]=x;
    } else {
      shift_reg[i]=shift_reg[i-1];
      acc+=shift_reg[i]*c[i];
    }
  }
  *y=acc;
}
```

Műveletek



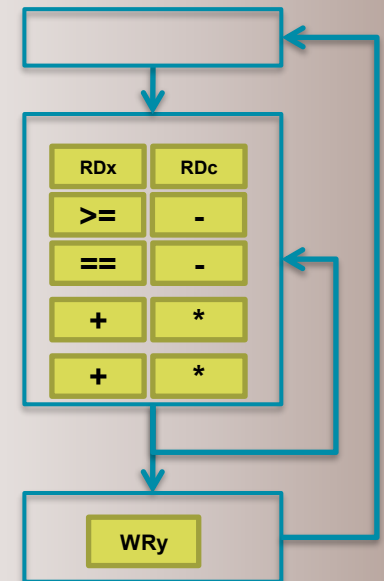
Vezérlés

Finite State Machine (FSM) states



Vezérlés és adatutak

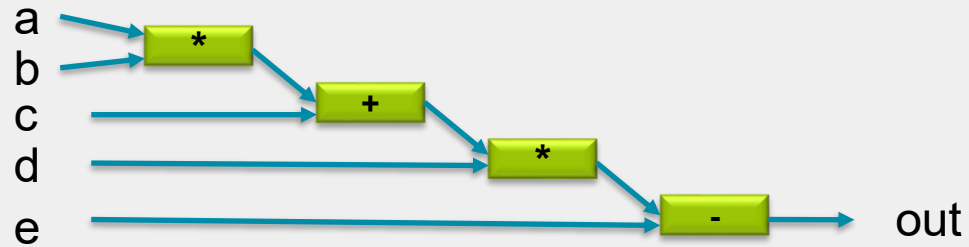
Control Dataflow



Ütemezés

- Alapja az adatfolyam gráf

```
void foo (  
...  
t1 = a * b;  
t2 = c + t1;  
t3 = d * t2;  
out = t3 - e;  
}
```



- Alacsony órajel frekvencia esetén



- Ha magasabb az előírt frekvencia



- Műveletvégzők jelterjedési idejének becslése

- HLS elég pontatlan, az implementált HW tipikusan alacsonyabb órajelet képes elérni

Allokáció

- A műveleteket a HLS könyvtárának HW elemeihez rendeli

- Megosztott vagy egyedi műveletvégző?



- Két szorzó kell, mert ugyanabban az órajelben végeznek műveletet

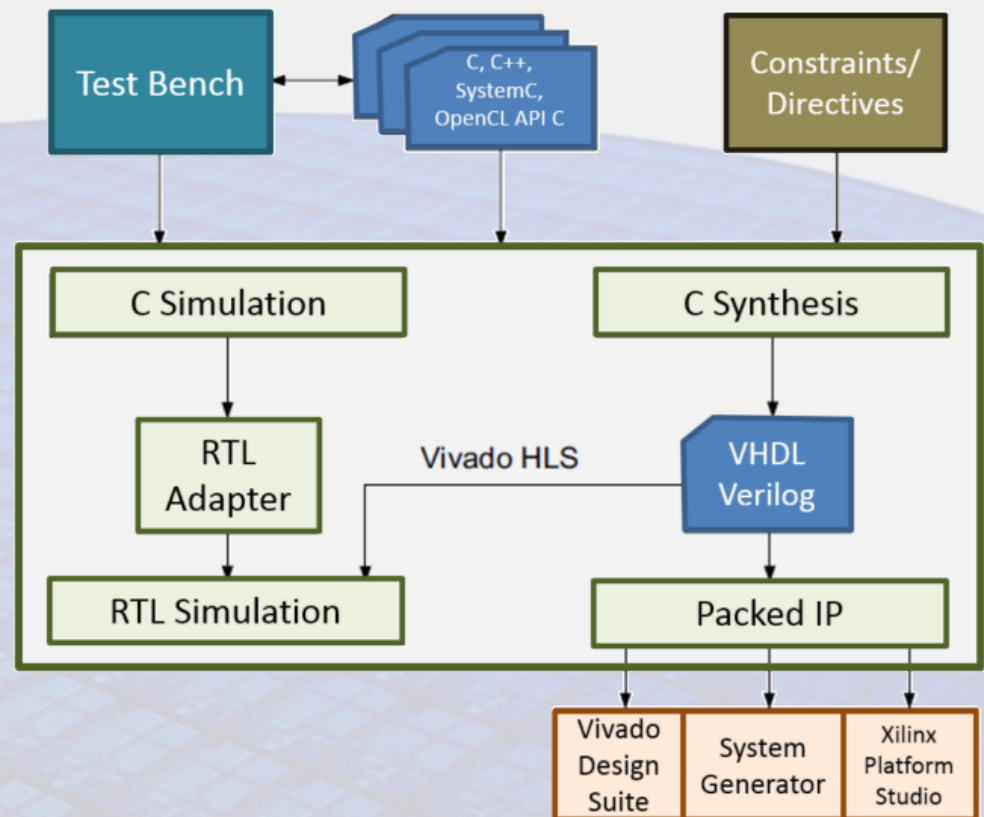
- Add-sub lehet közös – multiplexálás költsége



- Mind a szorzó, mind az Add-sub lehet közös

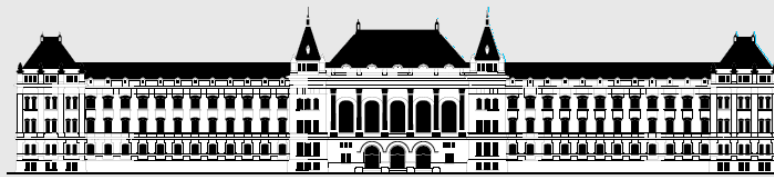
Validáció és verifikáció

- Az algoritmus C nyelvű → pre-szintézis algoritmus validáció C nyelvű környezetben
 - Nagyságrendekkel gyorsabb, mint a HDL szimuláció
- Poszt-szintézis RTL verifikáció
 - Az eredeti C testbench használatával



C testbench

- C main() függvény, amely meghívja a tesztelt (szintézisre szánt) függvényt
- **Automatikus tesztelés**
 - Ideális esetben a testbench az elvárt eredménnyel komaprálja a tesztelt függvény kimenetét
 - 0-val tér vissza egyezés esetén → sikeres validáció/verifikáció
 - Minden egyéb visszatérési érték hibát jelez



BUDAPESTI MŰSZAKI ÉS GAZDASÁGTUDOMÁNYI EGYETEM
VILLAMOSMÉRNÖKI ÉS INFORMATIKAI KAR
MESTERSÉGES INTELLIGENCIA ÉS RENDSZERTERVEZÉS TANSZÉK

Adattípusok

Adattípusok

- **Minden C adattípus használható**
 - (u)char, (u)short, (u)int, (u)long, float, double
- **Ez a legtöbb esetben pazarló → hardverben mindig a minimálisan szükséges adattípust használjuk**
- **ap_(u)int: egyedi szélességű egész típus**
- **ap_(u)fixed: fix pontos típus**
 - Fix pontos műveletvégzés megvalósítható egész változókkal is, de ez sokkal kényelmesebb
- **Fél-precíziós FP (FP16)**
 - Nincs tetszőleges pontosságú FP, bár sok AMD IP tud ilyet

ap_(u)int (1)

- Egyedi bitszélességű előjel nélküli és előjeles változó
 - 1...1024 bit
 - ap_int<W>
 - ap_uint<W>

```
#include <ap_int.h>
ap_uint<9> var0;    // 9 bites előjel nélküli
ap_int<18> var1;    // 18 bites előjeles
```


ap_(u)int (2)

■ Minden C++ aritmetikai operátor támogatott

Operations	
Arithmetic	+ - * / % ++ --
Logical	~ !
Bitwise	& ^
Relational	> < <= >= == !=
Assignment	*= /= %+= += -= <<= >>= &= ^= =

■ Típuskonverzió

Methods		Example
To integer	Convert to a integer type	res = var.to_int();
To unsigned integer	Convert to an unsigned integer type	res = var.to_uint();
To 64-bit integer	Convert to a 64-bit long long type	res = var.to_int64();
To 64-bit unsigned integer	Convert to an unsigned long long type	res = var.to_uint64();
To double	Convert to double type	res = var.double();

ap_(u)int (3)

▪ Egyéb metódusok

Methods		Example
Length	Returns the length of the variable.	res=var.length;
Concatenation	Concatenation low to high	res=var_hi.concat(var_lo); Or res= (var_hi,var_lo)
Range or Bit-select	Return a bit-range from high to low or a specific bit.	res=var.range(high bit,low bit); Or res=var[bit-number]
(n)and_reduce	(N)And reduce all bits.	bool t = var.and_reduce();
(n)or_reduce	(N)Or reduce all bits	bool t = var.or_reduce();
X(n)or_reduce	X(N)or reduce all bits	bool t = var.xor_reduce();
Reverse	Reverse the bits in the variable	var.reverse();
Test bit	Tests if a bit is true	bool t = var.test(bit-number)
Set bit value	Sets the value of a specific bit	var.set_bit(bit-number, value)
Set bit	Set a specific bit to one	var.set(bit-number);
Clear bit	Clear a specific bit to zero	var.clear(bit-number);
Invert Bit	Invert a specific bit	var.invert(bit-number);
Rotate right	Rotate the N-bits to the right	var.rrotate(N);
Rotate left	Rotate the N-bits to the left	var.lrotate(N);
Bitwise Invert	Invert all bits	var.b_not();
Test sign	Test if the sign is negative (return true)	bool t = var.sign();

ap_(u)fixed (1)

- **Fix pontos adattípus**

- Automatikus kettedespont illesztés különböző adattípusok esetén
- Kvantálás (törtrész bitszám csökkentés) és túlcsordulás (egészrész bitszám csökkentés) kezelés

[illegible]

ap_(u)fixed (2)

Fix pontos adattípus

- Teljes bitszám (W)
- Egészrész bitszám (I)
- Kvantálási (kerekítési) mód (Q)
- Túlcsordulás/szaturáció mód (O)
- (Szaturációs bitek száma)

ap_[u]fixed<W, I , Q, O , N>



Binary point : $W = I + B$

	Description	
W	Word length in bits	
I	The number of bits used to represent the integer value (the number of bits above the decimal point)	
Q	Quantization mode (modes detailed below) dictates the behavior when greater precision is generated than can be defined by the LSBs.	
	AP_Fixed Mode	Description
	AP_RND	Rounding to plus infinity
	AP_RND_ZERO	Rounding to zero
	AP_RND_MIN_INF	Rounding to minus infinity
	AP_RND_INF	Rounding to infinity
	AP_RND_CONV	Convergent rounding
	AP_TRN	Truncation to minus infinity
	AP_TRN_ZERO	Truncation to zero (default)
O	Overflow mode (modes detailed below) dictates the behavior when more bits are required than the word contains.	
	AP_Fixed Mode	Description
	AP_SAT	Saturation
	AP_SAT_ZERO	Saturation to zero
	AP_SAT_SYM	Symmetrical saturation
	AP_WRAP	Wrap around (default)
	AP_WRAP_SM	Sign magnitude wrap around
N	The number of saturation bits in wrap modes.	

ap_(u)fixed (3)

■ Operátorok

Operations	
Arithmetic	+ - * / % ++ --
Logical	~ !
Bitwise	& ^
Relational	> < <= >= == !=
Assignment	*= /= %= += -= <<= >>= &= ^= =

■ Típus konverzió

Methods		Example
To integer	Convert to a integer type	res = var.to_int();
To unsigned integer	Convert to an unsigned integer type	res = var.to_uint();
To 64-bit integer	Convert to a 64-bit long long type	res = var.to_int64();
To 64-bit unsigned integer	Convert to an unsigned long long type	res = var.to_uint64();
To double	Convert to double type	res = var.double();
To ap_int	Convert to an ap_int	res = var.to_ap_int();

ap_(u)fixed (4)

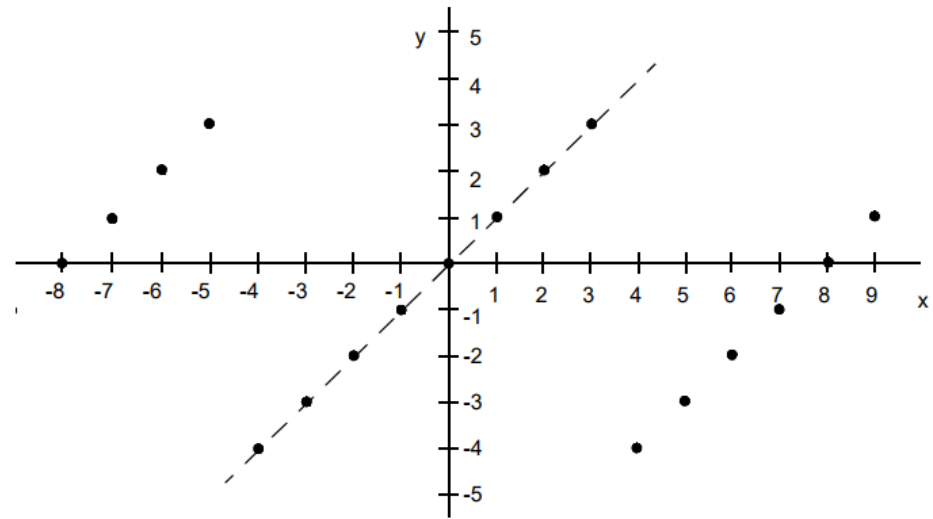
Kvantálás	
AP_RND	Kerekítés, a közéértéket $+\infty$ felé
AP_RND_ZERO	Kerekítés, a közéértéket nulla felé
AP_RND_MIN_INF	Kerekítés, a közéértéket $-\infty$ felé
AP_RND_INF	Kerekítés, a középső értéket negatív számok esetén $-\infty$, pozitív számok esetén $+\infty$ felé
AP_RND_CONV	Kerekítés, a közéértéket a kvantum legközelebbi páros többszöröse felé (egészre: legközelebbi páros szám)
AP_TRN	Csonkolás ($-\infty$ felé)
AP_TRN_ZERO	Csonkolás 0 felé

ap_(u)fixed (5)

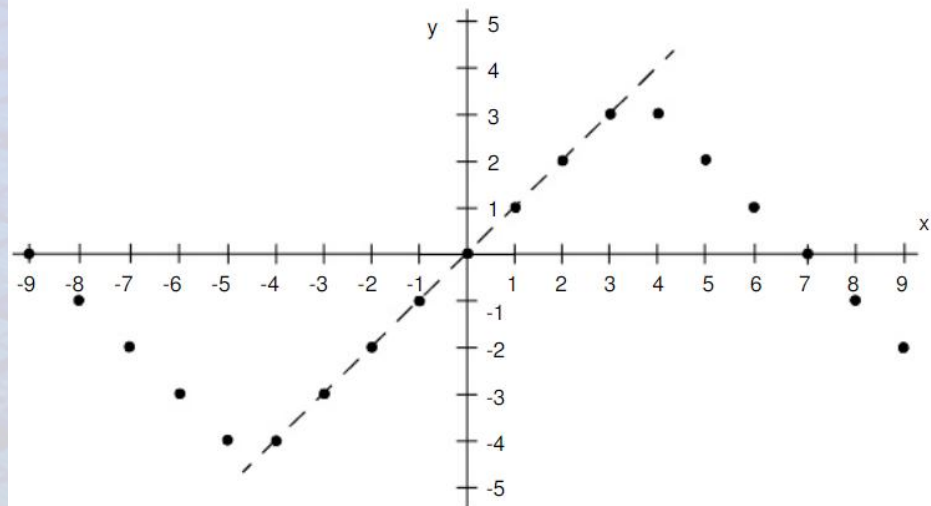
Túlcsordulás	
AP_SAT	Szaturáció
AP_SAT_ZERO	Túlcsordulás esetén nulla
AP_SAT_SYM	Szimmetrikus szaturáció
AP_WRAP	MSB bitek levágása
AP_WRAP_SM	MSB bitek levágása előjel megtartással

ap_(u)fixed (5)

- AP_WRAP



- AP_WRAP_SM



Lebegőpontos számok

- **IEEE-754 kompatibilis**

- Single-precision

- 32 bit: előjel, 23-bit mantissza, 8-bit exponens

- Double-precision

- 64 bit: előjel, 52-bit mantissza, 11-bit exponens

- Half-precision

- 16-bit: előjel, 10-bit mantissza, 5-bit exponens

Teljesítmény mérőszámok

- **Latency**

- Adott bemenethez tartozó kimenet hány órajel múlva készül el
- Ez a hardver késleltetése, órajelben számolva

- **Throughput**

- Adott kódrészlet (függvény, ciklus) hány órajelenként hajtható végre újból
- Throughput=1 → minden órajelben elkezdhető új végrehajtás → legnagyobb teljesítmény
- Throughput=X → X órajelenként hajtható végre

- **Initiation interval**

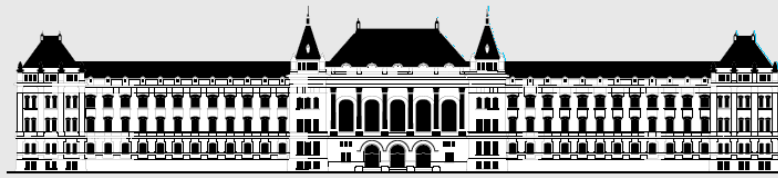
- Függvény hány órajelenként hívható meg

- **Iteration interval**

- Hány órajelenként indítható új iterációja egy ciklusnak

- **Iteration latency**

- Egy ciklus iteráció hány órajelet igényel (~ciklusmag latency)



BUDAPESTI MŰSZAKI ÉS GAZDASÁGTUDOMÁNYI EGYETEM
VILLAMOSMÉRNÖKI ÉS INFORMATIKAI KAR
MESTERSÉGES INTELLIGENCIA ÉS RENDSZERTERVEZÉS TANSZÉK

FIR szűrő shift regiszterrel

C kód

```
#include "ap_int.h"
#include "ap_fixed.h"

typedef ap_fixed<18, 1, AP_RND_CONV, AP_WRAP> coeff_t;
typedef ap_fixed<24, 1, AP_TRN, AP_WRAP> din_t;
typedef ap_fixed<18+24, 2, AP_TRN, AP_WRAP> mul_t;
typedef ap_fixed<18+24+7, 2+7, AP_TRN, AP_WRAP> accu_t;
typedef ap_fixed<24, 1, AP_TRN, AP_SAT> dout_t;

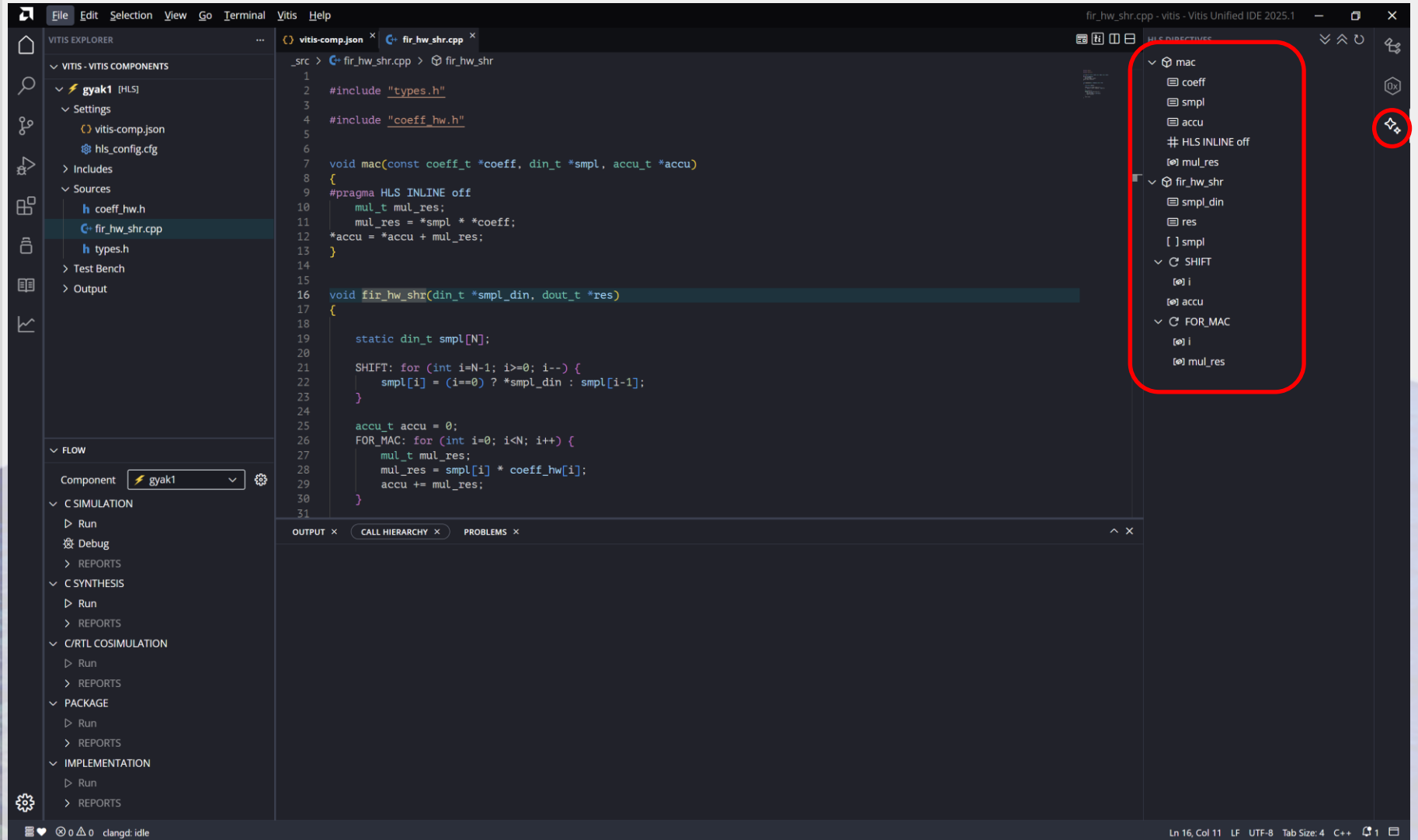
void fir_hw_shr(din_t *smp1_din, dout_t *res)
{
    static din_t smp1[N];
    mul_t mul_res;

    SHIFT: for (int i=N-1; i>=0; i--) {
        smp1[i] = (i==0) ? *smp1_din : smp1[i-1];
    }

    accu_t accu = 0;
    FOR_MAC: for (int i=0; i<N; i++) {
        mul_res = smp1[i] * coeff_hw[i];
        accu += mul_res;
    }

    *res = accu;
}
```

Vitis HLS GUI



Minden direktíva kikapcsolva

■ 100 ns órajel, szorzás LUT-ban

MODULES & LOOPS	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
▼ ● fir_hw_shr (2)	5.140E4		515		no	function	2	0	98	979	0
● _C SHIFT	2.560E4	2	-	128	no	loop					
● _C FOR_MAC	2.560E4	2	-	128	no	loop					

■ 5 ns órajel, szorzás LUT-ban

MODULES & LOOPS	ISSUE	SLACK	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
▼ ● fir_hw_shr (2)	⚠	-1.87	6.166E3		899		no	function	2	0	203	991	0
● _C SHIFT			2.637E3	3	-	128	no	loop					
● _C FOR_MAC			3.516E3	4	-	128	no	loop					

■ 5 ns órajel, szorzás DSP blokkal

MODULES & LOOPS	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
▼ ● fir_hw_shr (2)	5.130E3		1,027		no	function	2	1	126	287	0
● _C SHIFT	1.920E3	3	-	128	no	loop					
● _C FOR_MAC	3.200E3	5	-	128	no	loop					

Erőforrások

- **Alapesetben minden C tömb legalább 1 BRAM**
 - Lehet több is, ha a tömb nem fér el egy BRAM-ban
 - Mert szélesebb
 - Mert mélyebb
- **Szorzás és/vagy MAC művelethez alapvetően DSP blokkot használ**
 - Minden művelethez beállítható, hogy milyen implementációt szeretnénk (BIND_OP)
 - Amennyiben nincs párhuzamosítás, egy DSP blokk elegendő a 18x24 bites MAC művelethez

BIND_OP

- **Adott művelet implementációja. Paraméterek:**
 - Kimeneti változó (→ NE írjunk kilométeres sorokat)
 - Művelet típusa (pl. mul, madd)
 - Implementáció (fabric/DSP)

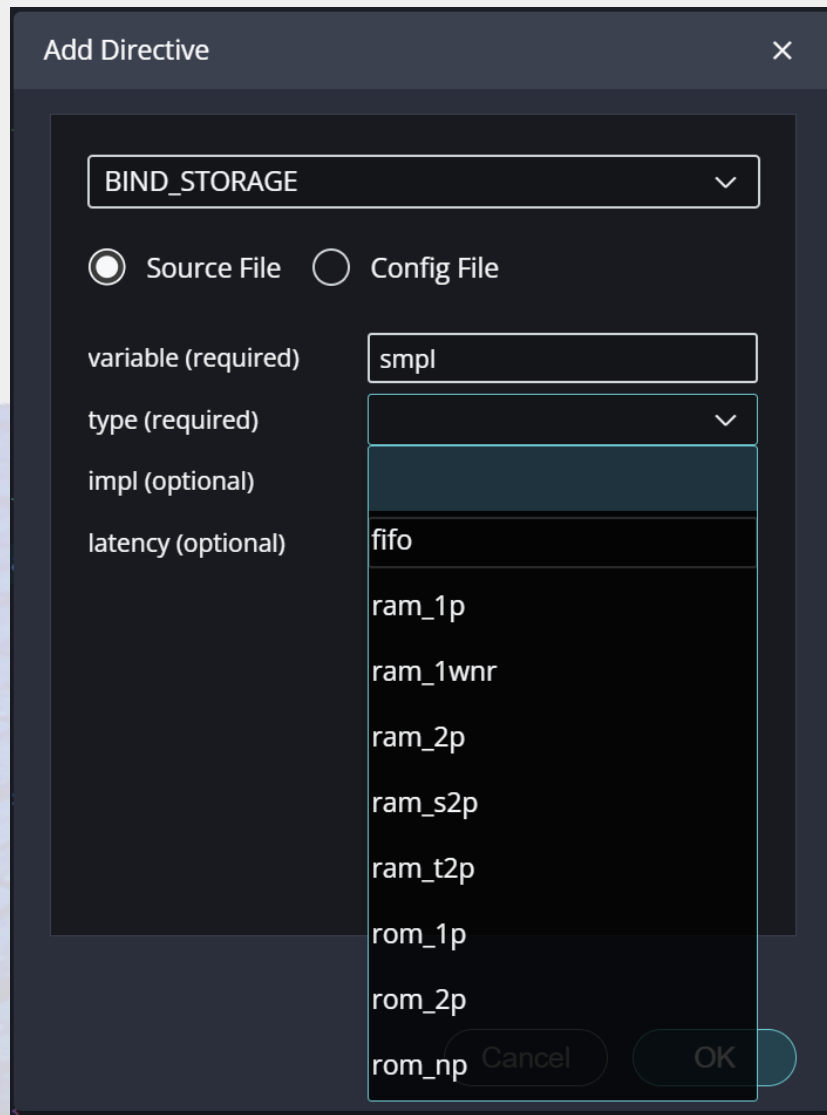


The screenshot shows a dark-themed 'Edit Directive' dialog box. At the top, there's a title bar with 'Edit Directive' and a close button. Below it, a dropdown menu is set to 'BIND_OP'. There are two radio buttons: 'Source File' (selected) and 'Config File'. Below these are four input fields: 'variable (required)' with 'mul_res', 'op (required)' with 'mul', 'impl (optional)' with 'fabric', and 'latency (optional)' with '0~4'. At the bottom right are 'Cancel' and 'OK' buttons.

Field	Value
Directive	BIND_OP
Source File	☒
Config File	☐
variable (required)	mul_res
op (required)	mul
impl (optional)	fabric
latency (optional)	0~4

BIND_STORAGE

- Többöt milyen hardver elemmel szeretnénk megvalósítani



The screenshot shows a dark-themed 'Add Directive' dialog box. At the top, the title 'Add Directive' is on the left and a close button 'X' is on the right. Below the title bar, there is a dropdown menu currently showing 'BIND_STORAGE'. Underneath this, there are two radio buttons: 'Source File' (which is selected) and 'Config File'. Below the radio buttons, there are four labels on the left and corresponding input fields on the right: 'variable (required)' with a text box containing 'smpl', 'type (required)' with a dropdown arrow, 'impl (optional)' with an empty text box, and 'latency (optional)' with a text box containing 'fifo'. A list of options is visible below the 'latency' input, including 'fifo', 'ram_1p', 'ram_1wnr', 'ram_2p', 'ram_s2p', 'ram_t2p', 'rom_1p', 'rom_2p', and 'rom_np'. At the bottom right, there are two buttons: 'Cancel' and 'OK'.

Add Directive

BIND_STORAGE

☒ Source File ☐ Config File

variable (required) smpl

type (required)

impl (optional)

latency (optional) fifo

ram_1p

ram_1wnr

ram_2p

ram_s2p

ram_t2p

rom_1p

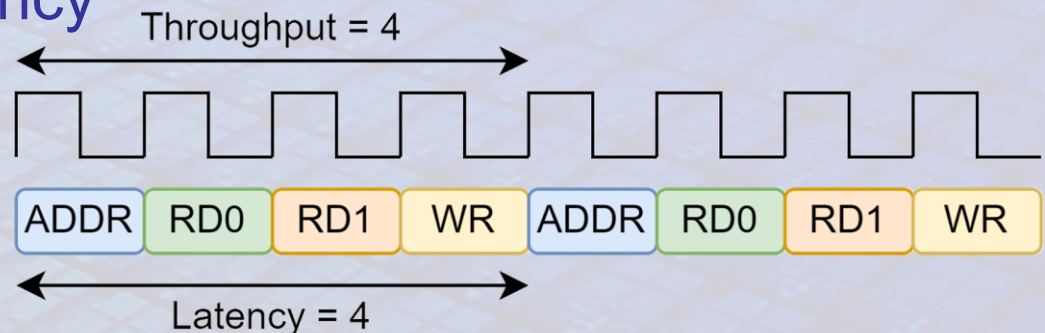
rom_2p

rom_np

Cancel OK

Időzítés (1)

- **Iteration latency: a ciklusmag “végrehajtási ideje” órajelben. Függ:**
 - A ciklusmagban végzett művelet teljes jelterjedési idejétől → azaz a leírt műveletektől, a használt adattípusoktól és a használt FPGA-tól
 - Az igényelt működési órajeltől → ennek megfelelő számú részre bontja a logikát regiszterek közbeiktatásával
- **Ciklus végrehajtás ideje**
 - Minden optimalizáció nélkül szekvenciális → iteráció szám * iteration latency



Időzítés (2)

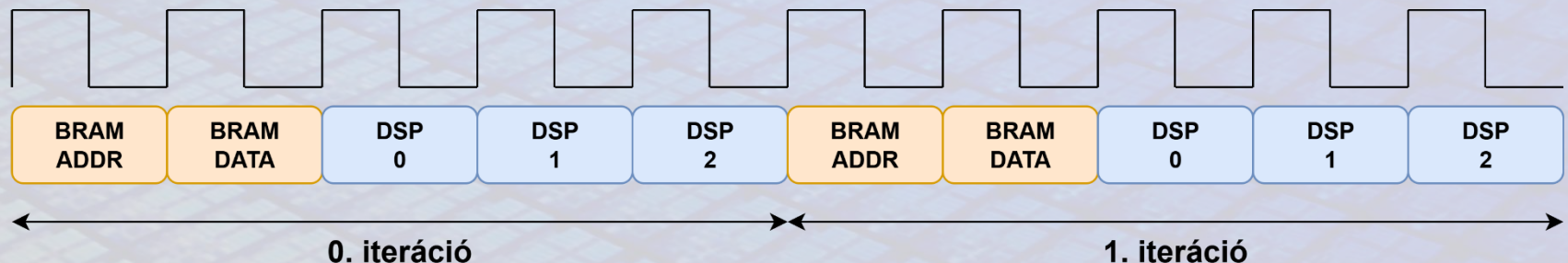
- **SHIFT ciklus**

- Tömb $\rightarrow$ BRAM
- A BRAM szinkron olvasású $\rightarrow$ olvasás minimum 2 órajel: cím + adat
- Teljes ciklus: iteráció szám * iteration latency
 - 100 ns: $128 \cdot 2$ órajel
 - 5 ns: $128 \cdot 3$ órajel

Időzítés (3)

■ MAC ciklus

- BRAM olvasás miatt minimum 2 órajel
- A ciklusmag késleltetése függ az igényelt órajeltől és a MAC művelet implementációjától
 - → 100 ns + LUT esetben a BRAM kimenetére csatlakozó kombinációs logika → összesen 2 órajel
 - → 5 ns esetben a MAC műveletvégzőt is több részre bontja regiszterekkel
 - LUT esetben rosszul, nem teljesíti az időzíteni kritériumot
 - DSP-nél a használt alapbeállítás: 3 regiszter szint



Időzítés (4)

- **Függvény: Initiation interval**
 - “Hány órajelenként hívható meg újra”
- **Teljes függvény (5 ns + DSP)**
 - Az egymás utáni ciklusok végrehajtása időben egy más után
 - SHIFT ciklus: $128 \cdot 3$ órajel = 384
 - MAC ciklus: $128 \cdot 5$ órajel = 640
 - Összesen: 1024 órajel
 - Plusz a vezérlő állapotgép overhead (néhány órajel)

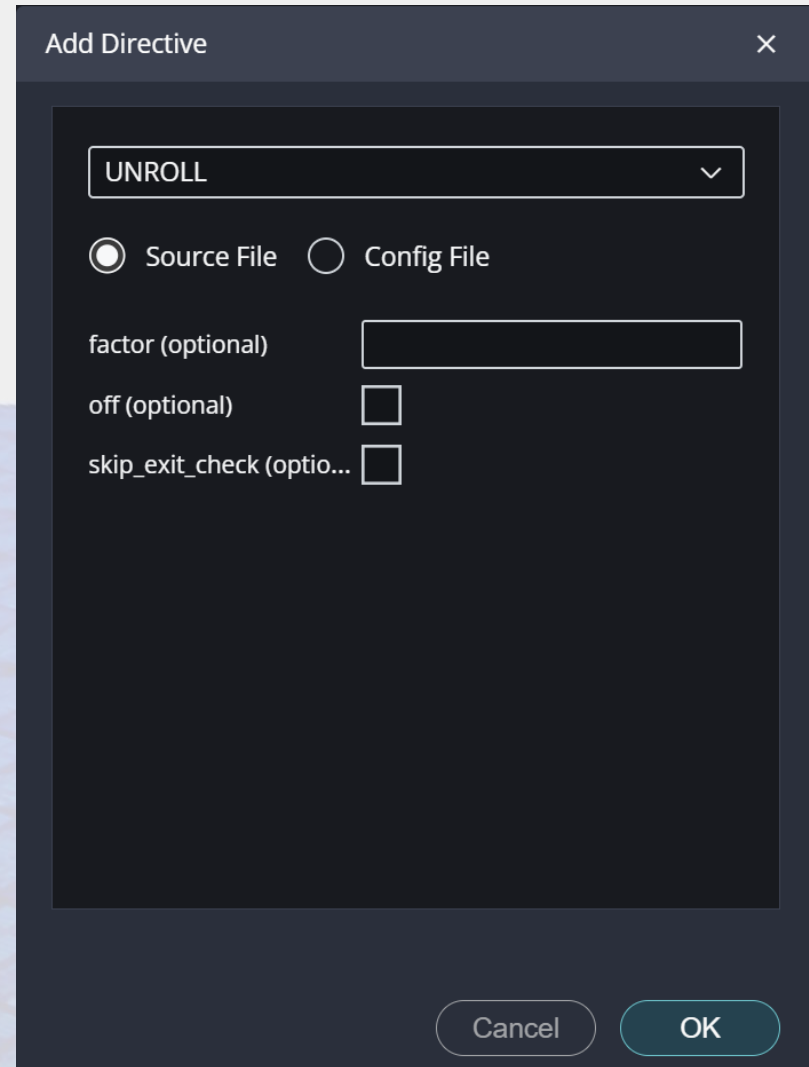
MODULES & LOOPS	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
▼ ● fir_hw_shr (2)	5.130E3		1,027		no	function	2	1	126	287	0
● _C SHIFT	1.920E3	3	-	128	no	loop					
● _C FOR_MAC	3.200E3	5	-	128	no	loop					

SHIFT ciklus gyorsítása

- Egy shift regiszter egy órajel alatt képes shiftelni
 - A kódban ciklus van → ehhez ezt ki kell teríteni → UNROLL
 - C tömb alapesetben BRAM → regiszterben szeretnénk → ARRAY_PARTITION

UNROLL

- **Ciklus kiterítése**
 - Az iterációk párhuzamosan hajtódnak végre
 - Többszörözi a végrehajtóegységeket → többszörös erőforrásigénnyel járhat
- **Lehet részleges UNROLL is (factor)**
 - Pl. factor=2 két iterációt hajt végre párhuzamosan



The screenshot shows a dark-themed dialog box titled "Add Directive" with a close button (X) in the top right corner. Inside the dialog, there is a dropdown menu currently showing "UNROLL". Below the dropdown are two radio buttons: "Source File" (which is selected) and "Config File". Underneath these are three optional settings, each with a label and a corresponding input field or checkbox: "factor (optional)" with a text input field, "off (optional)" with a checkbox, and "skip_exit_check (optio..." with a checkbox. At the bottom of the dialog are two buttons: "Cancel" and "OK".

ARRAY_PARTITON (1)

- **Tömb particionálása több logikai egységre**
 - Egy-egy logikai egység a méretétől függően lehet több fizikai memória!
- **Paraméterek**
 - Variable: a C tömb változója
 - Dim: melyik dimenzió szerint
 - Factor: hány fizikai egységre particionáljon
 - Type: hogyan particionáljon



The screenshot shows a dark-themed 'Add Directive' dialog box. At the top, there's a dropdown menu set to 'ARRAY_PARTITION'. Below it are two radio buttons: 'Source File' (selected) and 'Config File'. The form contains several input fields: 'variable (required)' with 'smpl', 'dim (optional)' with '1', 'factor (optional)' which is empty, 'off (optional)' with an unchecked checkbox, and 'type (optional)' with a dropdown menu showing 'block' (selected), 'cyclic', and 'complete'. At the bottom right are 'Cancel' and 'OK' buttons.

Field	Value
Directive	ARRAY_PARTITION
Source File / Config File	Source File
variable (required)	smpl
dim (optional)	1
factor (optional)	
off (optional)	☐
type (optional)	block

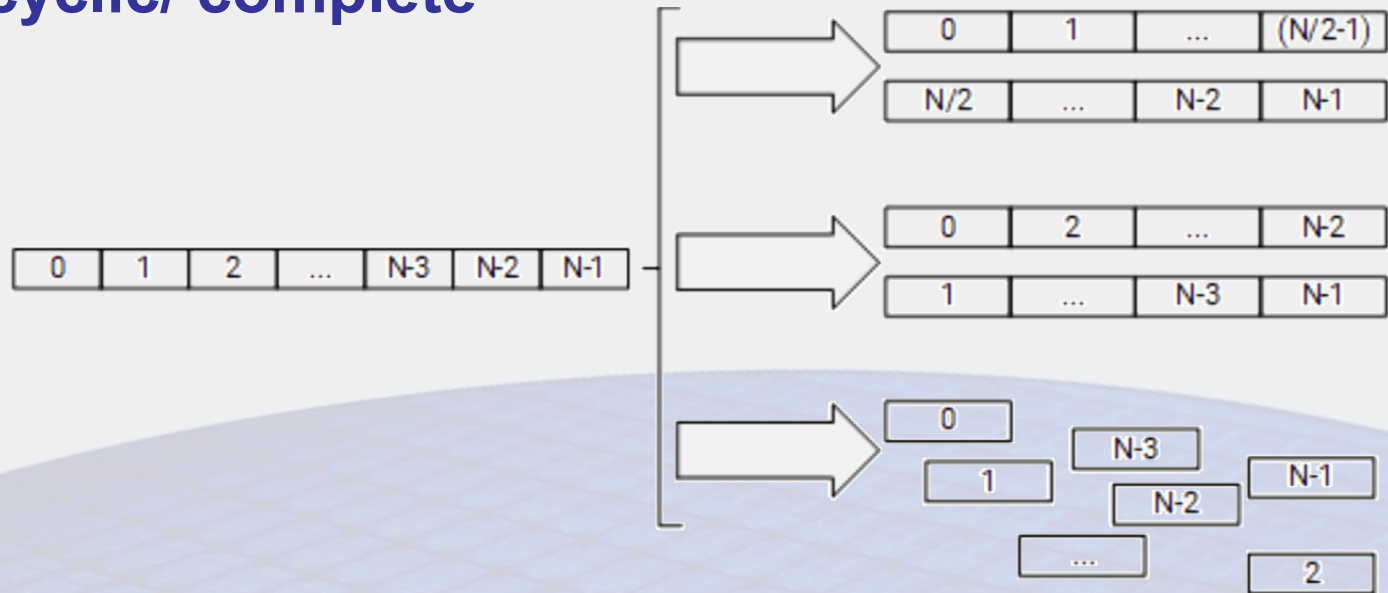
ARRAY_PARTITON (2)

■ Block/cyclic/ complete

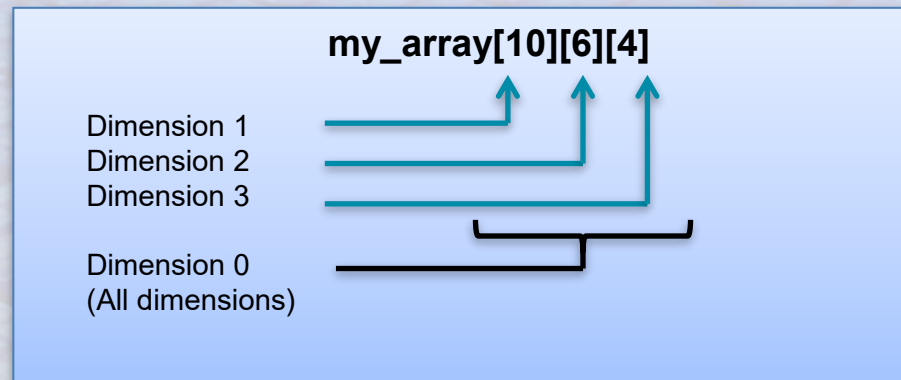
block

cyclic

complete

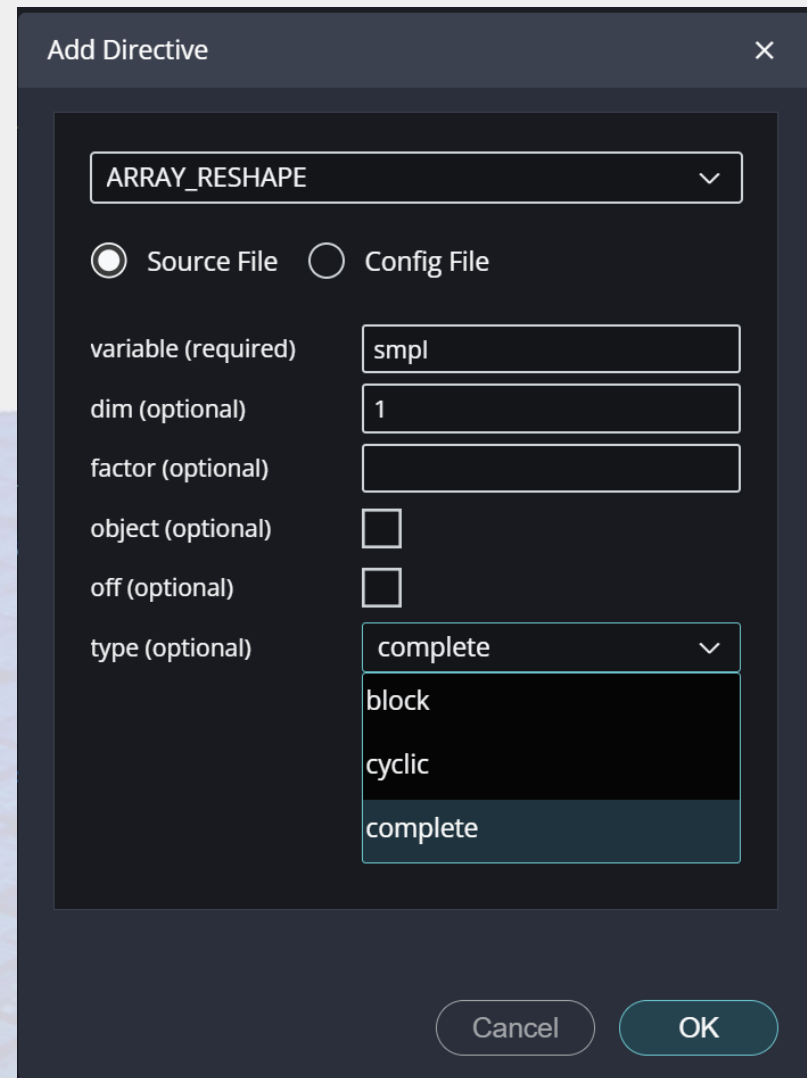


■ Dimenzió



ARRAY_RESHAPE (1)

- Tömb “átformázása” egy új tömbbé
 - Particionál
 - Majd összefűz
- Paraméterek
 - Ugyanaz, mint a particionálás esetében



The screenshot shows a dark-themed 'Add Directive' dialog box. At the top, the title bar says 'Add Directive' with a close button. Below the title bar, there is a dropdown menu currently showing 'ARRAY_RESHAPE'. Underneath, there are two radio buttons: 'Source File' (which is selected) and 'Config File'. Below these are several input fields: 'variable (required)' with the text 'smpl', 'dim (optional)' with the text '1', 'factor (optional)' which is empty, 'object (optional)' with an unchecked checkbox, 'off (optional)' with an unchecked checkbox, and 'type (optional)' with a dropdown menu showing 'complete'. The 'type' dropdown is open, showing a list of options: 'block', 'cyclic', and 'complete' (which is highlighted). At the bottom right, there are two buttons: 'Cancel' and 'OK'.

Add Directive

ARRAY_RESHAPE

☒ Source File ☐ Config File

variable (required)

dim (optional)

factor (optional)

object (optional) ☐

off (optional) ☐

type (optional)

block

cyclic

complete

Cancel OK

ARRAY_RESHAPE (2)

- Factor=2

array1[N]

0	1	2	...	N-3	N-2	N-1
---	---	---	-----	-----	-----	-----

block

array4[N/2]

MSB	N/2	...	N-2	N-1
LSB	0	1	...	(N/2-1)

array2[N]

0	1	2	...	N-3	N-2	N-1
---	---	---	-----	-----	-----	-----

cyclic

array5[N/2]

MSB	1	...	N-3	N-1
LSB	0	2	...	N-2

array3[N]

0	1	2	...	N-3	N-2	N-1
---	---	---	-----	-----	-----	-----

complete

array6[1]

MSB	N-1
	N-2
	...
	1
LSB	0

UNROLL + PARTITION

- A kiterített ciklus eltűnt az időzítésből

MODULES & LOOPS	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
▼ ● fir_hw_shr (1)	3.205E3		642		no	function	1	1	6,203	868	0
● _C FOR_MAC	3.200E3	5	-	128	no	loop					

- FF szám jelentősen megnőtt

- Remélhetnénk, hogy ez a tényleges implementáció után LUT SRL lesz, de sajnos nem.
- Megoldás: ap_shift_reg

AP_SHIFT_REG

```
void fir_hw_shr_2(din_t *smp1_din, dout_t *res)
{
    static ap_shift_reg<din_t, N> smp1;
    din_t smp1_rd;

    smp1.shift(*smp1_din, N-1);

    accu_t accu = 0;
    MAC: for (int i=0; i<N; i++)
    {
        mul_t mul_res;
        mul_res = smp1.read(i) * coeff_hw[i];
        accu += mul_res;
    }

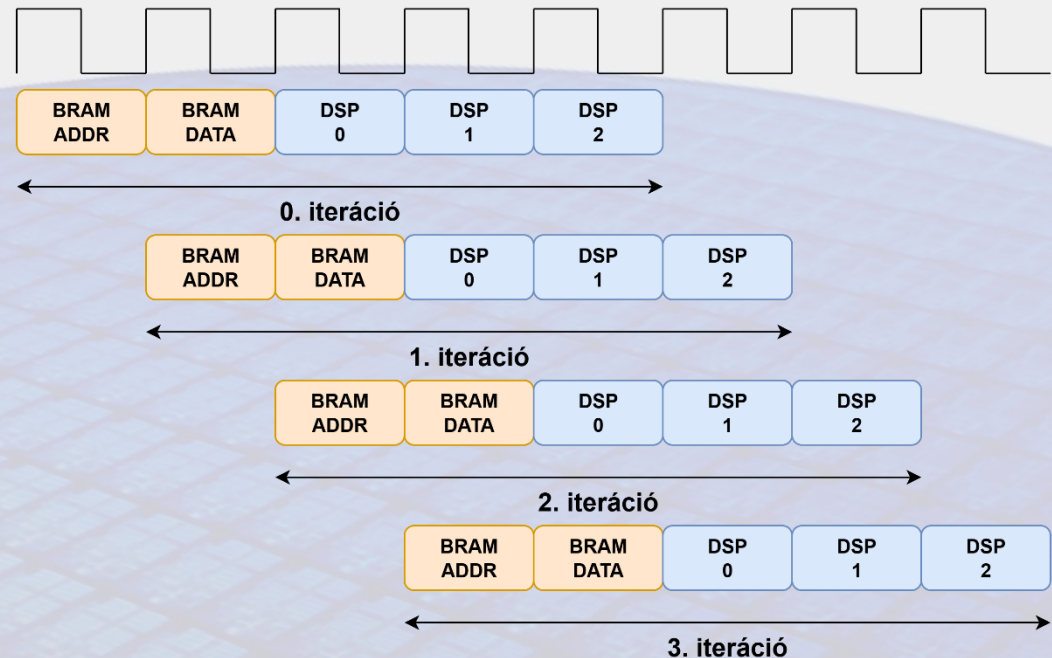
    *res = accu;
}
```

MAC ciklus optimalizálás

- Az igényelt órajeltől függő számú regiszter szint már megvan, de a végrehajtás NEM pipeline jellegű
- A pipeline végrehajtás az esetek jelentős részében nem növeli meg drasztikusan az erőforrás igényt

- NEM mindig igaz

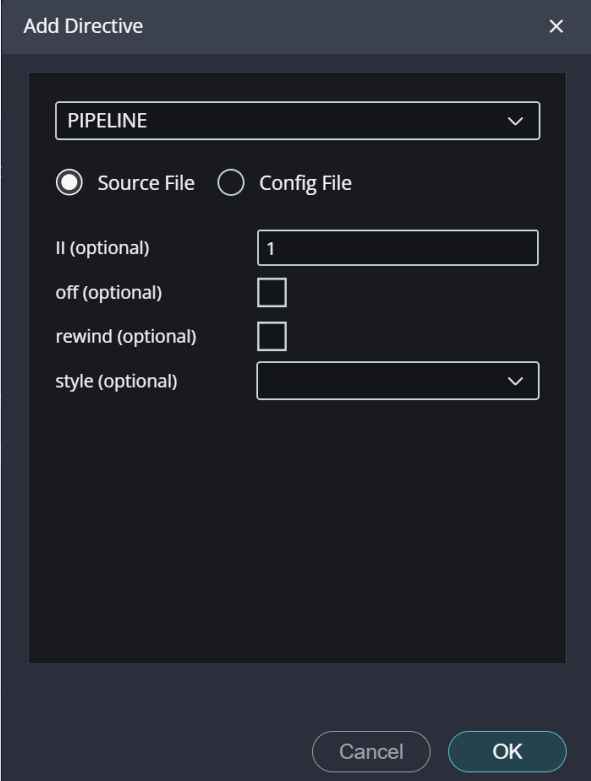
- **PIPELINE direktíva**



PIPELINE

■ Paraméterek

- II (initiation interval): az iterációk megkezdése közötti órajelek száma (1: minden órajelben új iteráció kezdődik)
- Rewind: A teljes ciklus újra indítható órajel veszteség nélkül
- Style
 - STP: stalled pipeline. Ha nincs új bemenet, a teljes pipeline áll.
 - FLP: Flushable. Már beléptetett adat akkor is végigmegy, ha nem érkezik új adat
 - FRP: Free-running, csak Dataflow régióban



The screenshot shows a dark-themed 'Add Directive' dialog box. At the top, there's a dropdown menu set to 'PIPELINE'. Below it are two radio buttons: 'Source File' (selected) and 'Config File'. Further down are four optional settings: 'II (optional)' with a text input containing '1', 'off (optional)' with an unchecked checkbox, 'rewind (optional)' with an unchecked checkbox, and 'style (optional)' with a dropdown menu. At the bottom right are 'Cancel' and 'OK' buttons.

Pipeline MAC

- Végrehajtási idő: iteráció szám * iteration latency

MODULES & LOOPS	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
▼ ● fir_hw_shr (1)	665.000		134		no	function	1	1	6,249	877	0
🔧 FOR_MAC	655.000	5	1	128	yes	loop					

- Ez már egy elfogadható 1-MAC FIR szűrő
 - Kb ugyanaz, mint a tavalyi HDL gyakorlaton megvalósított, csak a mintatár shiftregiszterben van

PIPELINE

- Amennyiben egy nem-pipeline implementációban ugyanazt a műveletvégzőt más-más órajelben az adatfolyam gráf más-más műveletére használja a hardver (“resource sharing”), akkor a pipeline használata járhat jelentősebb erőforrás növekedéssel

Egymásba ágyazott ciklus

- **Belső ciklus pipeline**
 - Hatékony, kis erőforrásigény
- **Külső ciklus pipeline**
 - A belső ciklus(oka)t megpróbálja UNROLL-olni
 - Konstans iteráció szám!
 - Erőforrás igény!

```
void foo(in1[ ][ ], in2[ ][ ]) {  
...  
  L1:for(i=1;i<N;i++) {  
    L2:for(j=0;j<M;j++) {  
#pragma HLS PIPELINE  
      out[i][j] = in1[i][j] +  
                  in2[i][j];  
    }  
  }  
}
```

1 összeadó, 3 load/store

```
void foo(in1[ ][ ], in2[ ][ ]) {  
...  
  L1:for(i=1;i<N;i++) {  
#pragma HLS PIPELINE  
    L2:for(j=0;j<M;j++) {  
      out[i][j] = in1[i][j] +  
                  in2[i][j];  
    }  
  }  
}
```

Unroll L2
M összeadó, 3M load/store

```
void foo(in1[ ][ ], in2[ ][ ]) {  
#pragma HLS PIPELINE  
...  
  L1:for(i=1;i<N;i++) {  
    L2:for(j=0;j<M;j++) {  
      out[i][j] = in1[i][j] +  
                  in2[i][j];  
    }  
  }  
}
```

Unroll L1 és L2
N*M összeadó, 3(N*M) load/store

Nagyobb pontosság

```
#include "ap_int.h"
#include "ap_fixed.h"

typedef ap_fixed<32, 1, AP_RND_CONV, AP_WRAP> coeff_t;
typedef ap_fixed<24, 1, AP_TRN, AP_WRAP> din_t;
typedef ap_fixed<32+24, 2, AP_TRN, AP_WRAP> mul_t;
typedef ap_fixed<32+24+7, 2+7, AP_TRN, AP_WRAP> accu_t;
typedef ap_fixed<24, 1, AP_TRN, AP_SAT> dout_t;

void fir_hw_shr(din_t *smp1_din, dout_t *res)
{
    NEM VÁLTOZIK
}
```

MODULES & LOOPS	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
✓  fir_hw_shr (1)	670.000		135		no	function	1	2	6,479	980	0
 FOR_MAC	660.000	6	1	128	yes	loop					

- Megfelel a várakozásoknak
 - Két DSP blokk a 32x24 bites szorzáshoz

Részben párhuzamos MAC

- Amennyiben az előbbi teljesítmény nem elegendő → többszörözzük meg a műveletvégzőt → MAC UNROLL
- 2x UNROLL

MODULES & LOOPS	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
✓  fir_hw_shr (1)	355.000		72		no	function	1	2	6,551	813	0
 FOR_MAC	345.000	7	1	64	yes	loop					

- 4x UNROLL

MODULES & LOOPS	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
✓  fir_hw_shr (1)	200.000		41		no	function	0	4	7,003	945	0
 FOR_MAC	190.000	8	1	32	yes	loop					

- A BRAM-t automatikusan particionálta
- Iteration latency nő – párhuzamosan számolt részeredmények összeadása
- Egyre rosszabb “hatásfok” – állapotgép overhead, egyre hosszabb pipeline “kiürülés”

Párhuzamos MAC

■ Teljes UNROLL

MODULES & LOOPS	LATENCY(NS)	INTERVAL	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
● fir_hw_shr	40.000	9	no	function	0	128	15,677	2,935	0

- Pocsék eredmény, az elvárt 1 helyett 9 az interval

■ Top-level függvény pipeline → maga után vonja a MAC unroll-t

MODULES & LOOPS	LATENCY(NS)	INTERVAL	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
● fir_hw_shr	40.000	1	yes	function	0	128	16,509	2,900	0

- Ez megfelel az elvárásainknak

Részben párhuzamos MAC (2)

- A részleges MAC unroll nem vezetett túl jó eredményre
- Pipeline-osítsuk a top-level függvényt
- Majd adjunk meg limitet a használható szorzók számára (ALLOCATION)
- Limit=64

MODULES & LOOPS	LATENCY(NS)	INTERVAL	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
● fir_hw_shr	35.000	2	yes	function	0	64	14,906	6,615	0

- Limit=4

MODULES & LOOPS	SLACK	LATENCY(NS)	INTERVAL	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
● fir_hw_shr	-0.51	198.000	32	yes	function	0	4	6,953	5,961	0

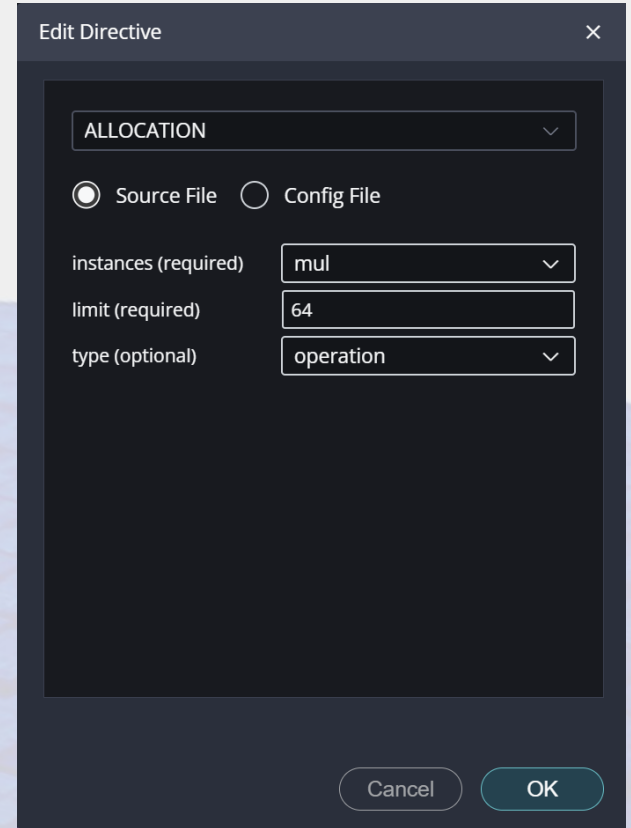
ALLOCATION

- **Paraméterek**

- Type: operation vagy function
- Limit: felhasználható mennyiség
- Instances: a művelet vagy a függvény típusa

- **Megjegyzés**

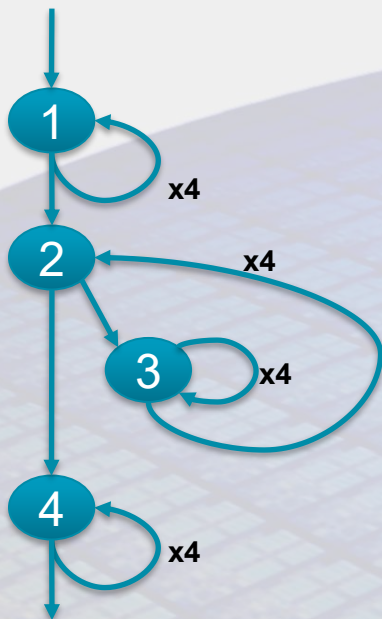
- Rossz szintaxissal generálja a GUI



The screenshot shows a dark-themed 'Edit Directive' dialog box. At the top, the title bar says 'Edit Directive' with a close button. Below the title bar, there is a dropdown menu currently set to 'ALLOCATION'. Underneath, there are two radio buttons: 'Source File' (which is selected) and 'Config File'. Below these are three input fields: 'instances (required)' with a dropdown menu showing 'mul', 'limit (required)' with a text input containing '64', and 'type (optional)' with a dropdown menu showing 'operation'. At the bottom right, there are two buttons: 'Cancel' and 'OK'.

LOOP_FLATTEN (1)

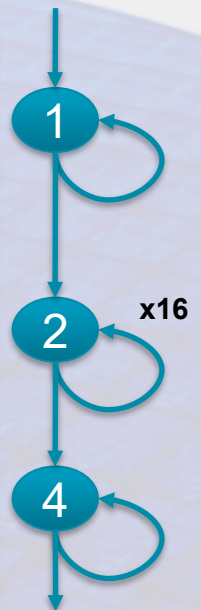
- **Egymásba ágyazott ciklusok**
 - A vezérlő állapotgép egy órajelet igényel a külső-belső ciklus(ok) közötti átmenethez
- **LOOP_FLATTEN összevonja a ciklusokat, így az kevesebb órajelet igényel**



```
void foo_top (...) {  
    ...  
    L1: for (i=3;i>=0;i--) {  
        [Loop body L1 ]  
    }  
  
    L2: for (i=3;i>=0;i--) {  
        L3: for (j=3;j>=0;j--) {  
            [Loop body L3 ]  
        }  
    }  
  
    L4: for (i=3;i>=0;i--) {  
        [Loop body L4 ]  
    }  
}
```



```
void foo_top (...) {  
    ...  
    L1: for (i=3;i>=0;i--) {  
        [Loop body L1 ]  
    }  
  
    L2: for (k=15,k>=0;k--) {  
        [Loop body L3 ]  
    }  
  
    L4: for (i=3;i>=0;i--) {  
        [Loop body L1 ]  
    }  
}
```



LOOP_FLATTEN (2)

- Alapesetben bekapcsolva
- Csak az alábbi ciklus típusokra működik

- “Perfect loop”

- Csak a legbelső ciklusnak van tartalma
- Konstans iteráció szám

```
Loop_outer: for (i=3;i>=0;i--) {  
    Loop_inner: for (j=3;j>=0;j--) {  
        [Loop body]  
    }  
}
```

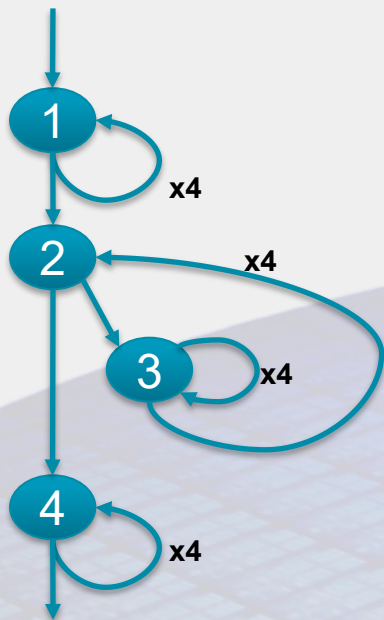
- “Semi-perfect loop”

- Minden más esetben kód módosítás szükséges

```
Loop_outer: for (i=3;i>N;i--) {  
    Loop_inner: for (j=3;j>=0;j--) {  
        [Loop body]  
    }  
}
```

LOOP_MERGE (1)

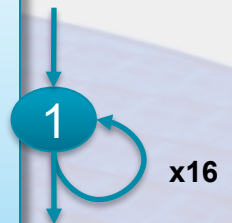
- Egymást követő ciklusok összevonása



```
void foo_top (...) {  
    ...  
    L1: for (i=3;i>=0;i--) {  
        [Loop body L1 ]  
    }  
    L2: for (i=3;i>=0;i--) {  
        L3: for (j=3;j>=0;j--) {  
            [Loop body L3 ]  
        }  
        LOOP_FLATTEN  
    }  
    L4: for (i=3;i>=0;i--) {  
        [Loop body L4 ]  
    }  
}
```



```
void foo_top (...) {  
    ...  
    L123: for (l=16,l>=0;l--) {  
        if (cond1)  
            [Loop body L1 ]  
        [Loop body L3 ]  
        if (cond4)  
            [Loop body L4 ]  
    }  
}
```

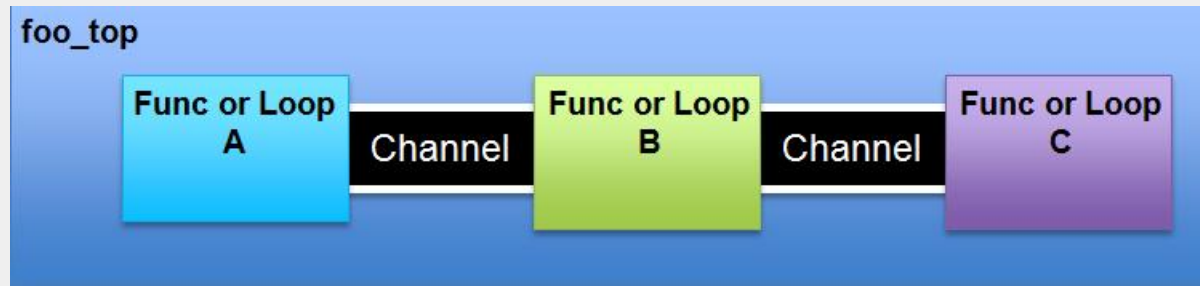


LOOP_MERGE (2)

- Ha változó az iteráció szám, akkor ugyanaz kell legyen
- Ha konstans az iteráció szám, akkor a maximum lesz az összevont ciklus iteráció száma
 - Lásd előző dia
- Változó és konstans iteráció számú ciklusok nem vonhatók össze

DATAFLOW (1)

- **Blokk szintű pipeline-osítás**
 - Blokk: függvény vagy ciklus



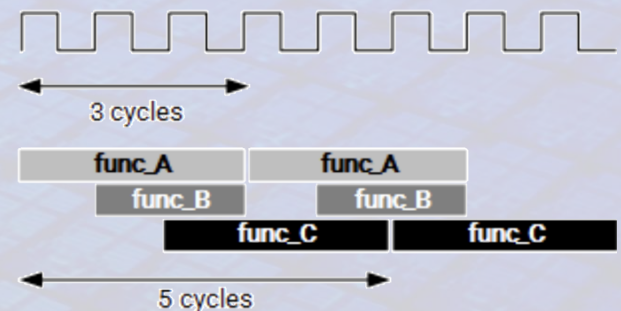
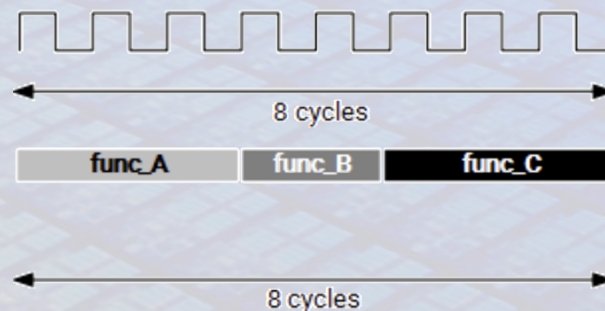
- **Channel: memória → van erőforrás igénye**
 - Függvény: ping-pong buffer
 - Ciklus: FIFO vagy ping-pong buffer

DATAFLOW (2)

- A blokkok párhuzamosan hajtódnak végre amennyiben az adatfüggőségek ezt megengedik
- Egy blokkban a feldolgozás azonnal megkezdődik, amint van bemeneti adat
- A blokkok között a tömbök elemenként kerülnek átadásra

```
void top (a,b,c,d) {  
    ...  
    func_A(a,b,i1);  
    func_B(c,i1,i2);  
    func_C(i2,d)  
  
    return d;  
}
```

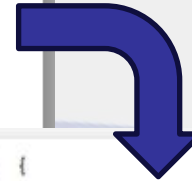
func_A
func_B
func_C



DATAFLOW (3)

- Szigorúan egy forrás – egy fogyasztó

```
void foo(int data_in[N], int scale, int data_out1[N], int data_out2[N]) {  
  
    int templ[N];  
    Loop1: for(int i = 0; i < N; i++) {  
        templ[i] = data_in[i] * scale;  
    }  
    Loop2: for(int j = 0; j < N; j++) {  
        data_out1[j] = templ[j] * 123;  
    }  
    Loop3: for(int k = 0; k < N; k++) {  
        data_out2[j] = templ[k] * 456;  
    }  
}
```

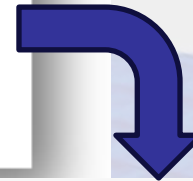


```
void Split (in[N], out1[N], out2[N]) {  
    // Duplicated data  
    L1:for(int i=1;i<N;i++) {  
        out1[i] = in[i];  
        out2[i] = in[i];  
    }  
}  
  
void foo(int data_in[N], int scale, int data_out1[N], int data_out2[N]) {  
  
    int templ[N], temp2[N], temp3[N];  
    Loop1: for(int i = 0; i < N; i++) {  
        templ[i] = data_in[i] * scale;  
    }  
    Split(templ, temp2, temp3);  
    Loop2: for(int j = 0; j < N; j++) {  
        data_out1[j] = temp2[j] * 123;  
    }  
    Loop3: for(int k = 0; k < N; k++) {  
        data_out2[j] = temp3[k] * 456;  
    }  
}
```

DATAFLOW (4)

- Adatút nem “ugorhat át” blokkot

```
void foo(int data_in[N], int scale, int data_out1[N], int data_out2[N]) {  
    int temp1[N], temp2[N], temp3[N];  
    Loop1: for(int i = 0; i < N; i++) {  
        temp1[i] = data_in[i] * scale;  
        temp2[i] = data_in[i] >> scale;  
    }  
    Loop2: for(int j = 0; j < N; j++) {  
        temp3[j] = temp1[j] + 123;  
    }  
    Loop3: for(int k = 0; k < N; k++) {  
        data_out[j] = temp2[k] + temp3[k];  
    }  
}
```



```
void foo(int data_in[N], int scale, int data_out1[N], int data_out2[N]) {  
    int temp1[N], temp2[N], temp3[N], temp4[N];  
    Loop1: for(int i = 0; i < N; i++) {  
        temp1[i] = data_in[i] * scale;  
        temp2[i] = data_in[i] >> scale;  
    }  
    Loop2: for(int j = 0; j < N; j++) {  
        temp3[j] = temp1[j] + 123;  
        temp4[j] = temp2[j];  
    }  
    Loop3: for(int k = 0; k < N; k++) {  
        data_out[j] = temp4[k] + temp3[k];  
    }  
}
```


PERFORMANCE (1)

- Top-level függvényre vagy ciklusokra
- Az elvárt teljesítményt adjuk meg

Add Directive

PERFORMANCE

☒ Source File ☐ Config File

target_ti (required)

1

unit (optional)

cycle

cycle

sec

Cancel

OK

PERFORMANCE (2)

- Jól hangzik, de nem váltja meg a világot

- Target=1

MODULES & LOOPS	TARGET TI(NS)	ESTIMATED TI(NS)	PERFORMANCE CONSTRAINT	LATENCY(NS)	INTERVAL	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
● fir_hw_shr	5.000	5.000	pragma	40.000	1	yes	function	0	128	16,509	2,900	0

- Megegyezik a top-level pipeline ii=1 verzióval

- Target=2

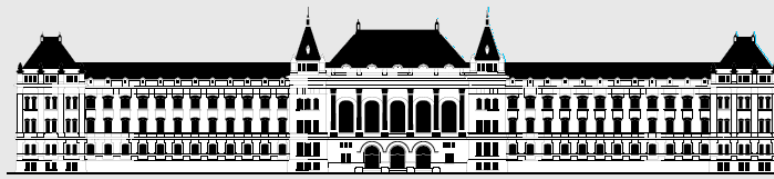
MODULES & LOOPS	TARGET TI(NS)	ESTIMATED TI(NS)	PERFORMANCE CONSTRAINT	LATENCY(NS)	INTERVAL	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
● fir_hw_shr	10.000	5.000	pragma	40.000	1	yes	function	0	128	16,509	2,900	0

- Nem ezt vártuk

- Target=32

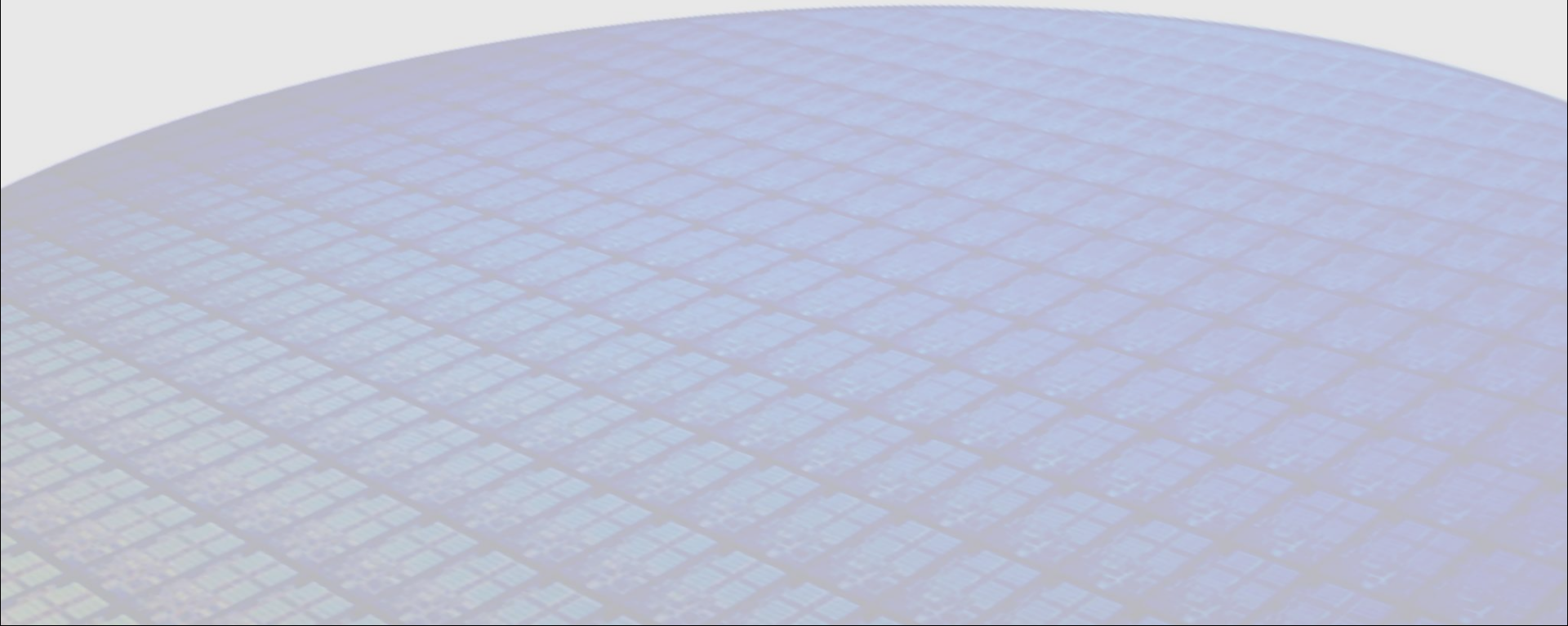
MODULES & LOOPS	TARGET TI(NS)	ESTIMATED TI(NS)	PERFORMANCE CONSTRAINT	LATENCY(NS)	INTERVAL	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
● fir_hw_shr	160.000	125.000	pragma	170.000	25	yes	function	0	128	14,660	4,528	0

- Lassú, de legalább nagy erőforrás-igényű ☺



BUDAPESTI MŰSZAKI ÉS GAZDASÁGTUDOMÁNYI EGYETEM
VILLAMOSMÉRNÖKI ÉS INFORMATIKAI KAR
MESTERSÉGES INTELLIGENCIA ÉS RENDSZERTERVEZÉS TANSZÉK

FIR szűrő cirkuláris bufferrel



FIR – cirkuláris buffer (1)

```
typedef ap_uint<7> addr_t;

void fir_hw_mem(din_t *smp1_din, dout_t *res)
{
    static din_t smp1_mem[N];

    static addr_t wr_addr;
    addr_t rd_addr;

    smp1_mem[wr_addr] = *smp1_din;
    rd_addr = wr_addr;
    wr_addr++;

    accu_t accu = 0;
    MAC: for (int i=(N-1); i>=0; i--)
    {
#pragma HLS PIPELINE
        mul_t mul_res;
        mul_res = smp1_mem[rd_addr] * coeff_hw[i];
        accu += mul_res;
        rd_addr--;
    }

    *res = accu;
}
```


FIR – cirkuláris buffer (2)

■ Az eredmény megfelel a várakozásoknak

MODULES & LOOPS	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
✓ ● fir_hw_mem (1)	680.000		137		no	function	2	1	198	322	0
✓ ● fir_hw_mem_Pipeline_MAC (1)	665.000		128		loop auto-rew	function	1	1	134	152	0
🔧 C MAC	655.000	5	1	128	yes	loop					

- 1-1 BRAM a két C tömbnek
 - Együttható 18 bit széles, 128 mély → 1 BRAM
 - Mintatár 24 bit széles, 128 mély → 1 BRAM
- 1 db DSP modul
 - 24x18 bites szorzás

FIR – cirkuláris buffer (3)

■ UNROLL factor=2

MODULES & LOOPS	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
✓ ● fir_hw_mem (1)	375.000		76		no	function	3	2	459	381	0
✓ ● fir_hw_mem_Pipeline_MAC (1)	360.000		64		loop auto-rew	function	1	2	395	202	0
⚙️ MAC	350.000	8	1	64	yes	loop					

NAME	USAGE	TYPE	BRAM	URAM	PRAGMA	VARIABLE	IMPL	LATENCY	BITWIDTH, DEPTH, BANKS
✓ ● fir_hw_mem (2)			3	-					
smpI_mem_U	ram_2p		2	-	no	smpI_mem	auto	1	24, 128, 1
✓ ● fir_hw_mem_Pipeline_MAC (2)			1	-					
coeff_hw_U	rom_2p		1	-	no	coeff_hw	auto	1	15, 128, 1
⚙️ MAC									

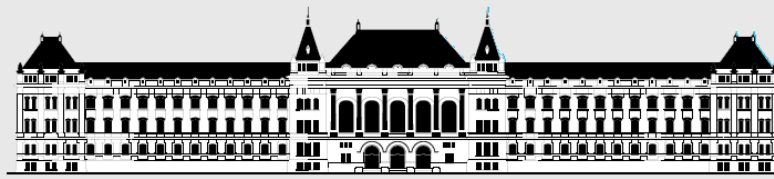
- 2 MAC → 2 DSP blokk
- 1 dual-port BRAM az együtthatóknak
- 2 dual-port BRAM a mintáknak
 - Ez elvben megoldható lenne 1 memóriával is

FIR – cirkuláris buffer (4)

■ UNROLL factor=4

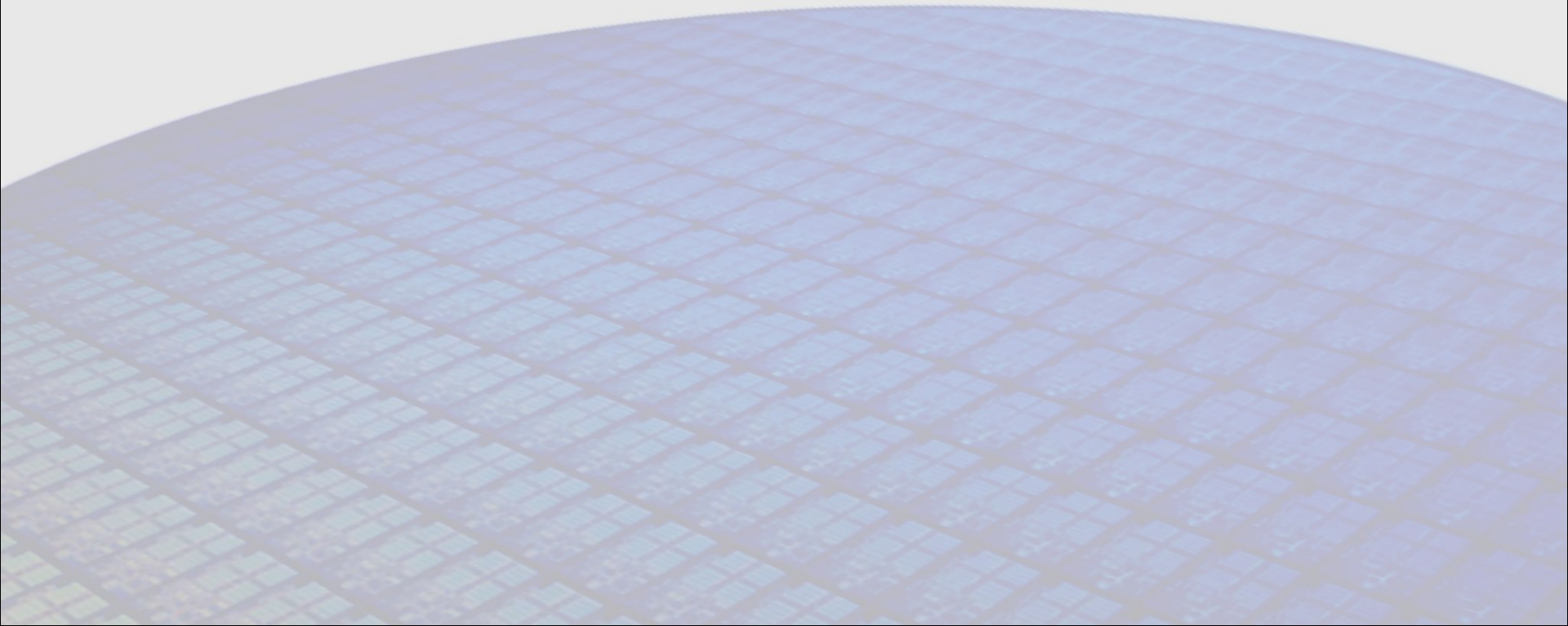
MODULES & LOOPS	ISSUE	VIOLATION TYPE	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
▼ ● fir_hw_mem (1)			375.000		76		no	function	4	4	719	783	0
▼ ● fir_hw_mem_Pipeline_MAC (1)			360.000		66		loop auto-rew	function	0	4	700	577	0
MAC	(ii)	Resource Limitation	350.000	9	2	32	yes	loop					

- A MAC ciklus nem valósítható meg iteration interval=1 opcióval
- “WARNING: [HLS 200-885] The II Violation in module 'fir_hw_mem_Pipeline_MAC' (loop 'MAC'): Unable to schedule 'load' operation 24 bit fir_hw_mem_ap_fixed due to limited memory ports (II = 1).”
- Ha belegondolunk, ez azért jogos...



BUDAPESTI MŰSZAKI ÉS GAZDASÁGTUDOMÁNYI EGYETEM
VILLAMOSMÉRNÖKI ÉS INFORMATIKAI KAR
MESTERSÉGES INTELLIGENCIA ÉS RENDSZERTERVEZÉS TANSZÉK

Blokk szintű interfészek

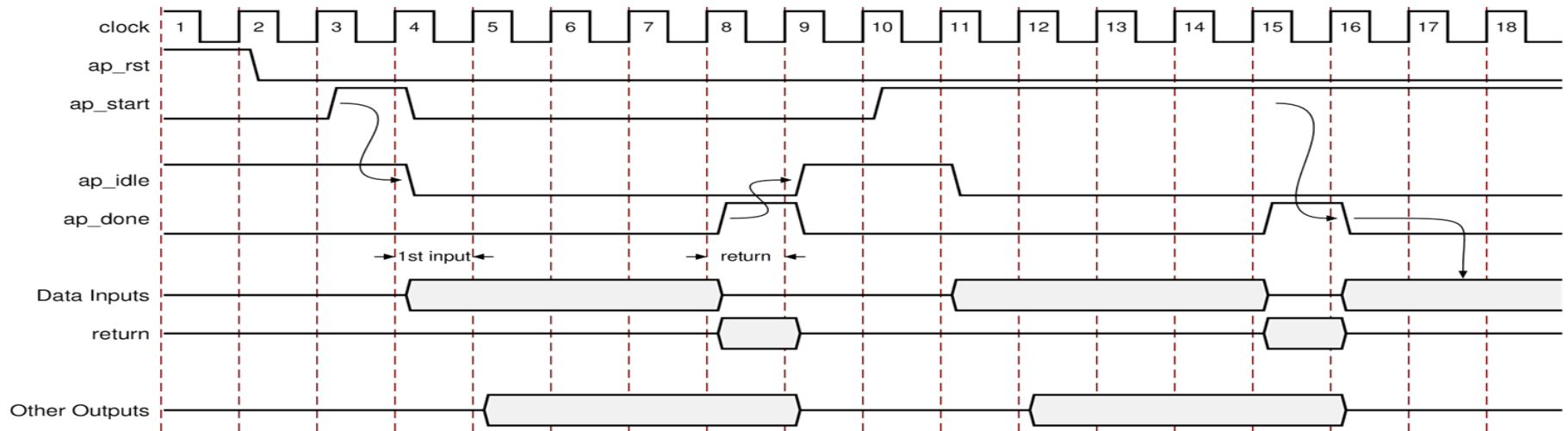


Blokk szintű interfészek (1)

- **Alapvető portok: órajel, reset**
 - Opcionálisan: clock enable
- **Visszatérési érték**
 - ap_return adat port
- **Handshake**
 - ap_start, ap_idle, ap_done
- **Top-level függvény argumentumai**
 - Ki- és bemeneti adat portok az adoot argumentum használatától függően
- **Opciók**
 - Kikapcsolva: ap_ctrl_none
 - Alapértelmezett handshake: ap_ctrl_hs
 - Kétirányú handshake: ap_ctrl_chain

Blokk szintű interfészek (2)

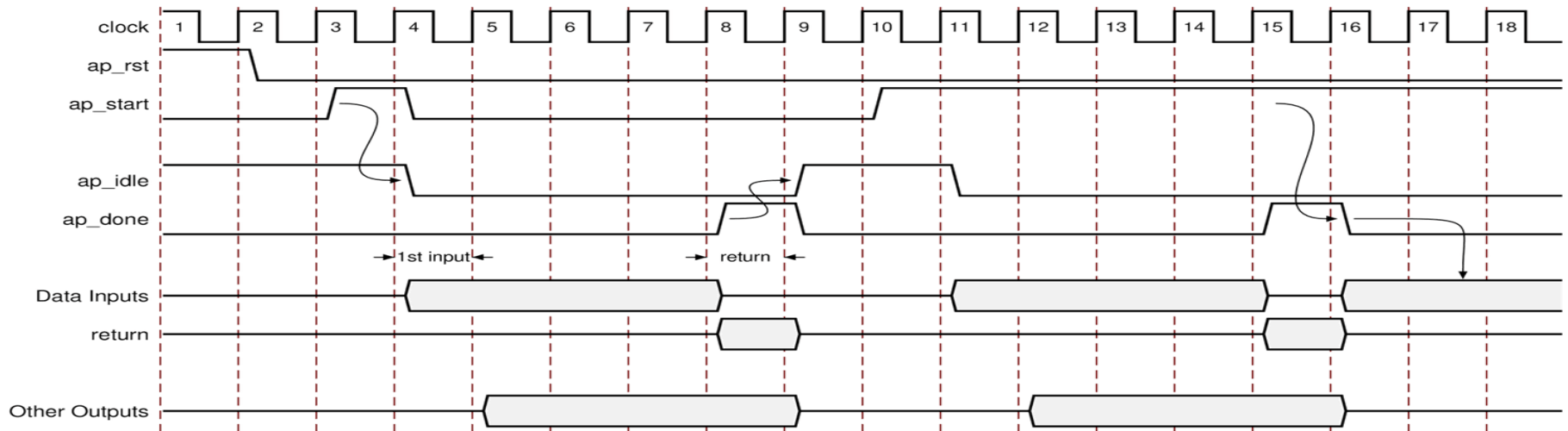
■ Blokk szintű handshake: ap_ctrl_hs



- Visszatérési érték akkor érvényes, amikor az `ap_done==1`
- Bemenetek olvasása bármikor történhet
- Kimenetek
 - Ha regisztrezett: `ap_done==1`
 - Egyébként nem tudjuk (→ port szintű handshake)

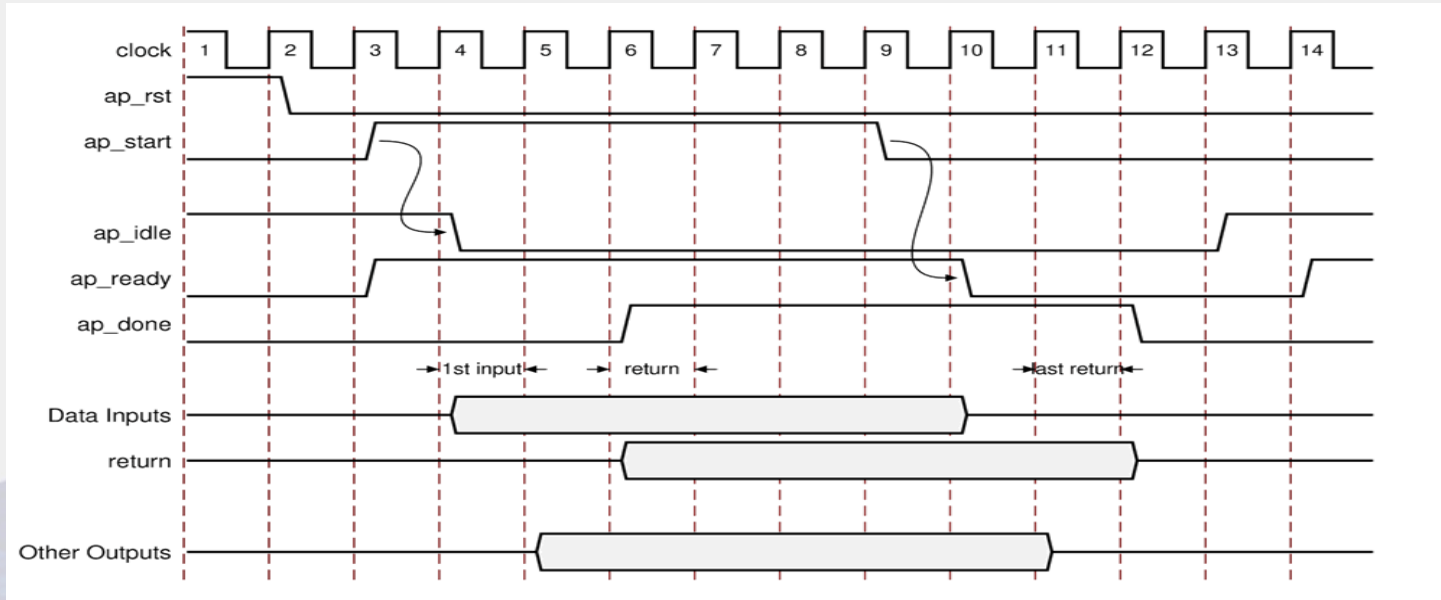
Blokk szintű interfészek (3)

■ Folyamatos újraindítás (ap_start=1)



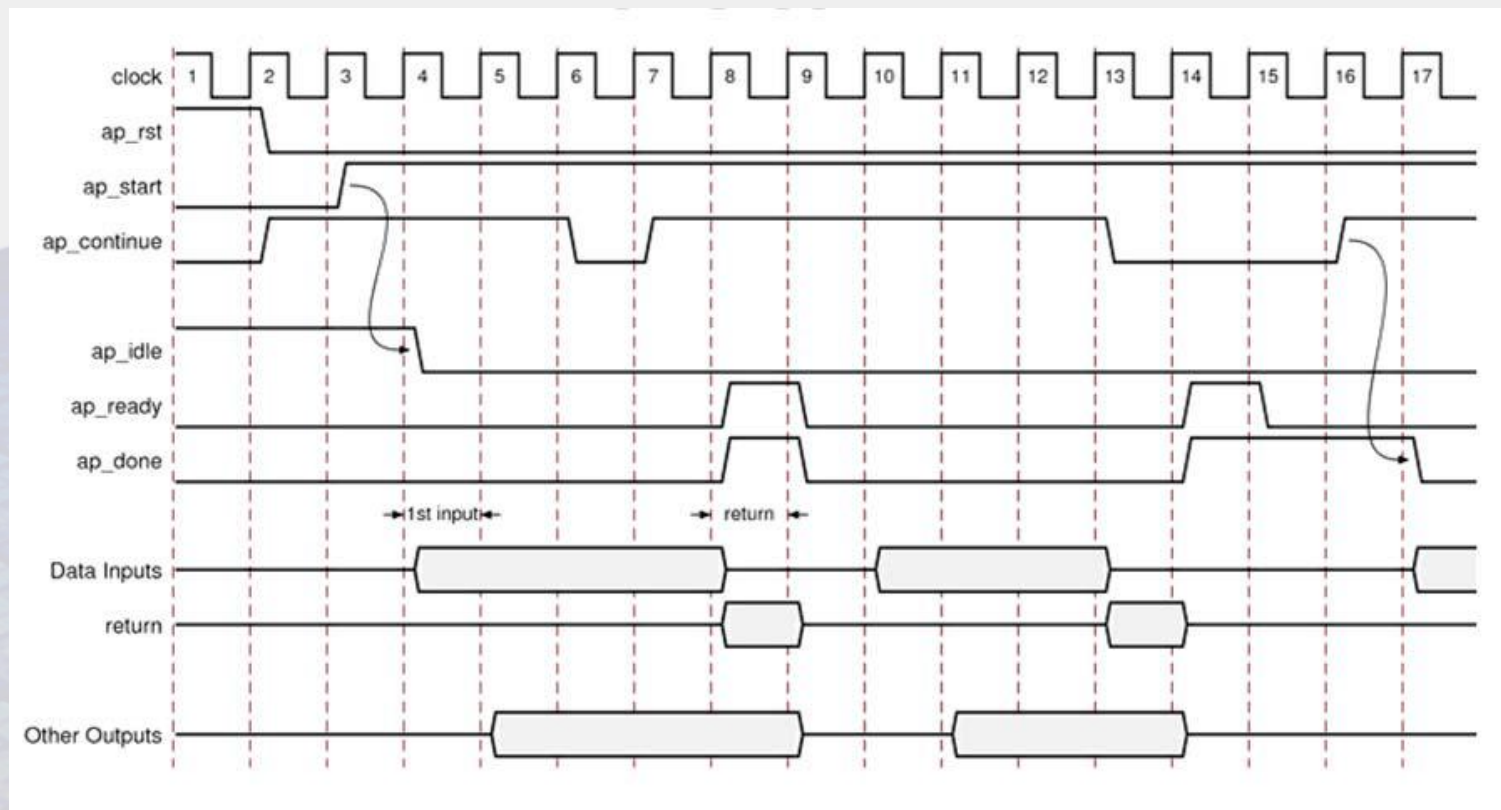
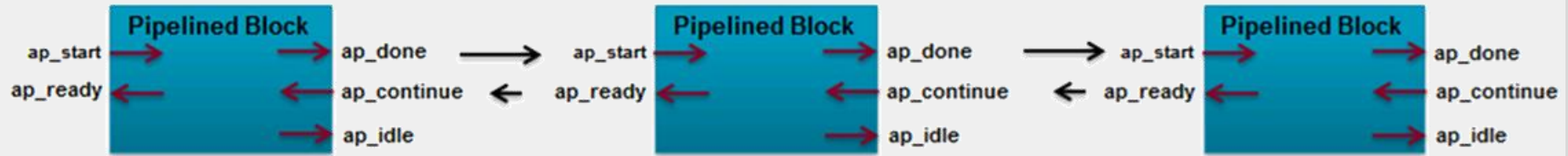
Blokk szintű interfészek (4)

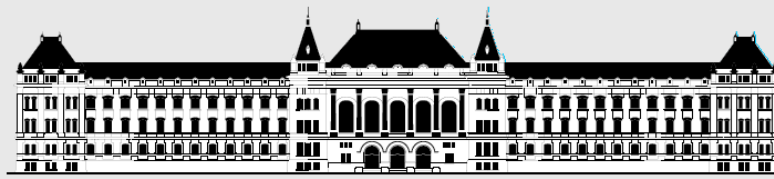
■ Pipeline II=1



Blokk szintű interfészek (5)

■ Kétirányú handshake: ap_ctrl_chain





BUDAPESTI MŰSZAKI ÉS GAZDASÁGTUDOMÁNYI EGYETEM
VILLAMOSMÉRNÖKI ÉS INFORMATIKAI KAR
MESTERSÉGES INTELLIGENCIA ÉS RENDSZERTERVEZÉS TANSZÉK

Port szintű interfészek

Interfész opciók

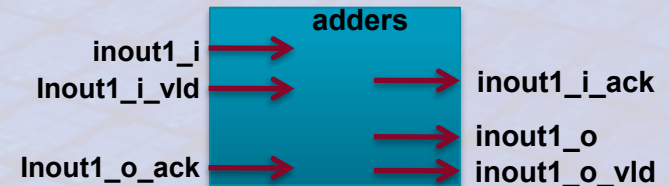
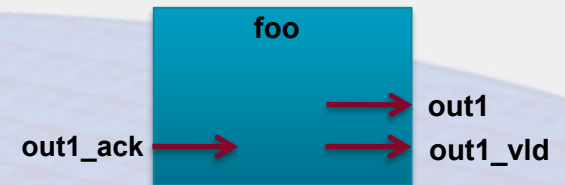
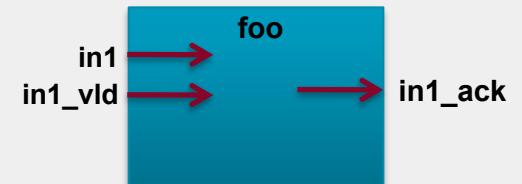
- **Interfész opciók argumentum típustól függően**

- D: default

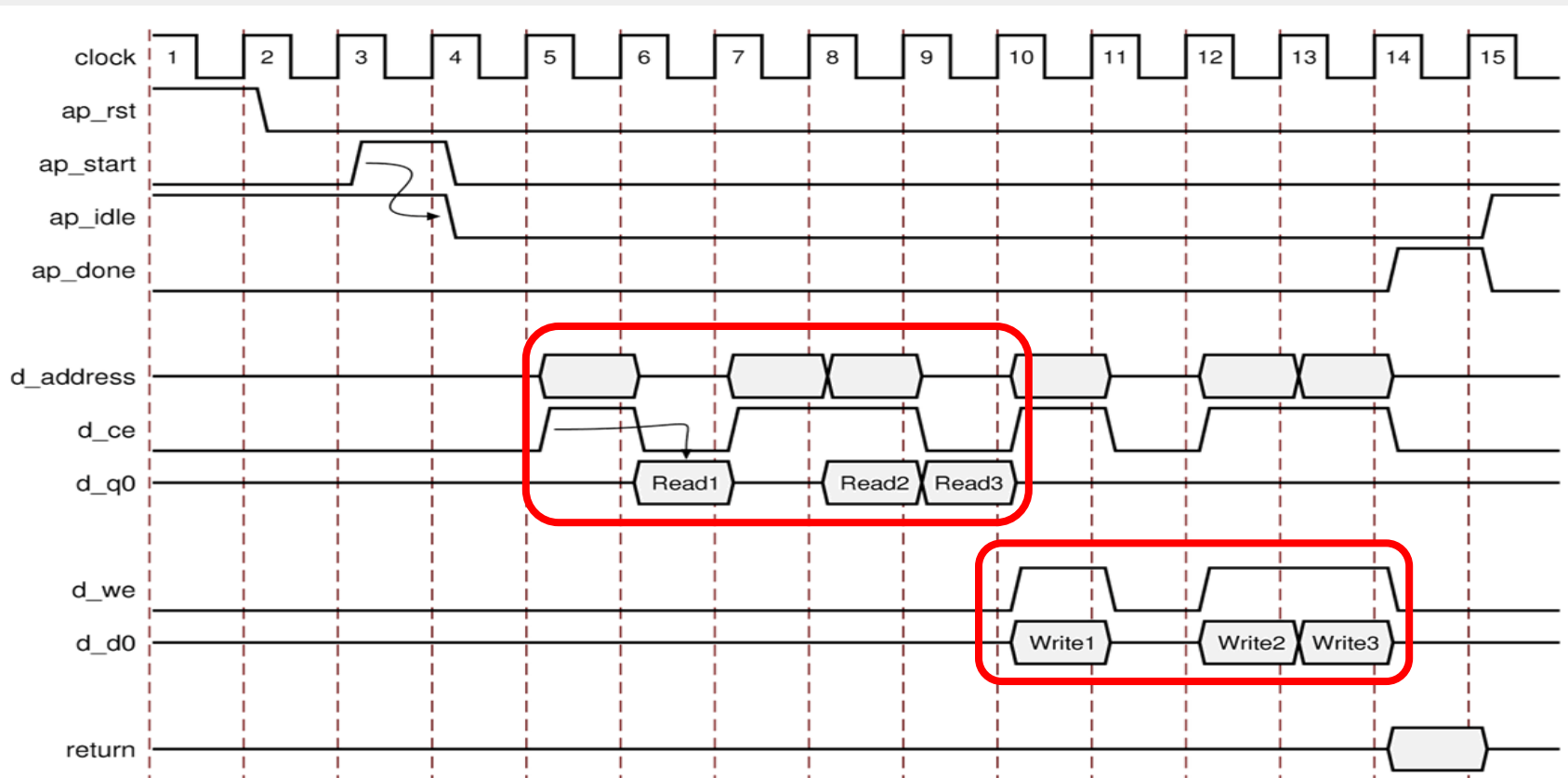
[illegible]

Handshake port

- **Kétirányú handshake: ap_hs**
- **Bemeneti port**
 - Csak olvasott argumentum
 - valid (be) és acknowledge (ki)
- **Kimeneti port**
 - Csak írt argumentum
 - valid (ki) és ack (be)
- **Kétirányú port**
 - Szétválasztott ki- és bemenet



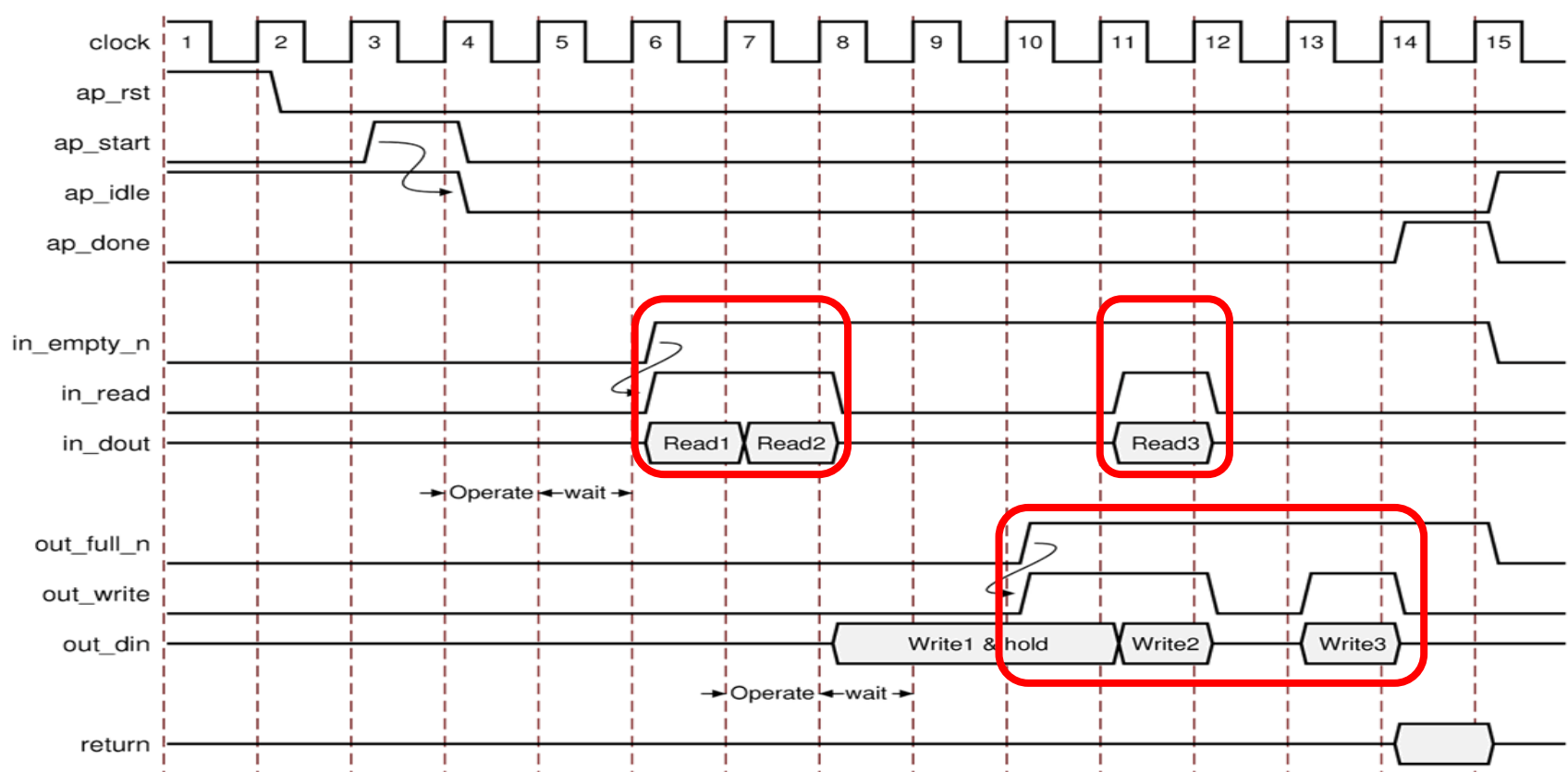
- **ap_memory**



FIFO port

■ ap_fifo

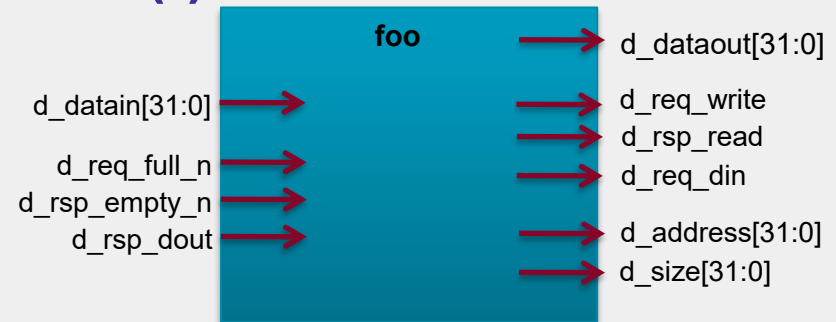
- Egy port egyirányú



Bus I/O

- **Támogatja az egyszeres és a burst(!) hozzáférést**

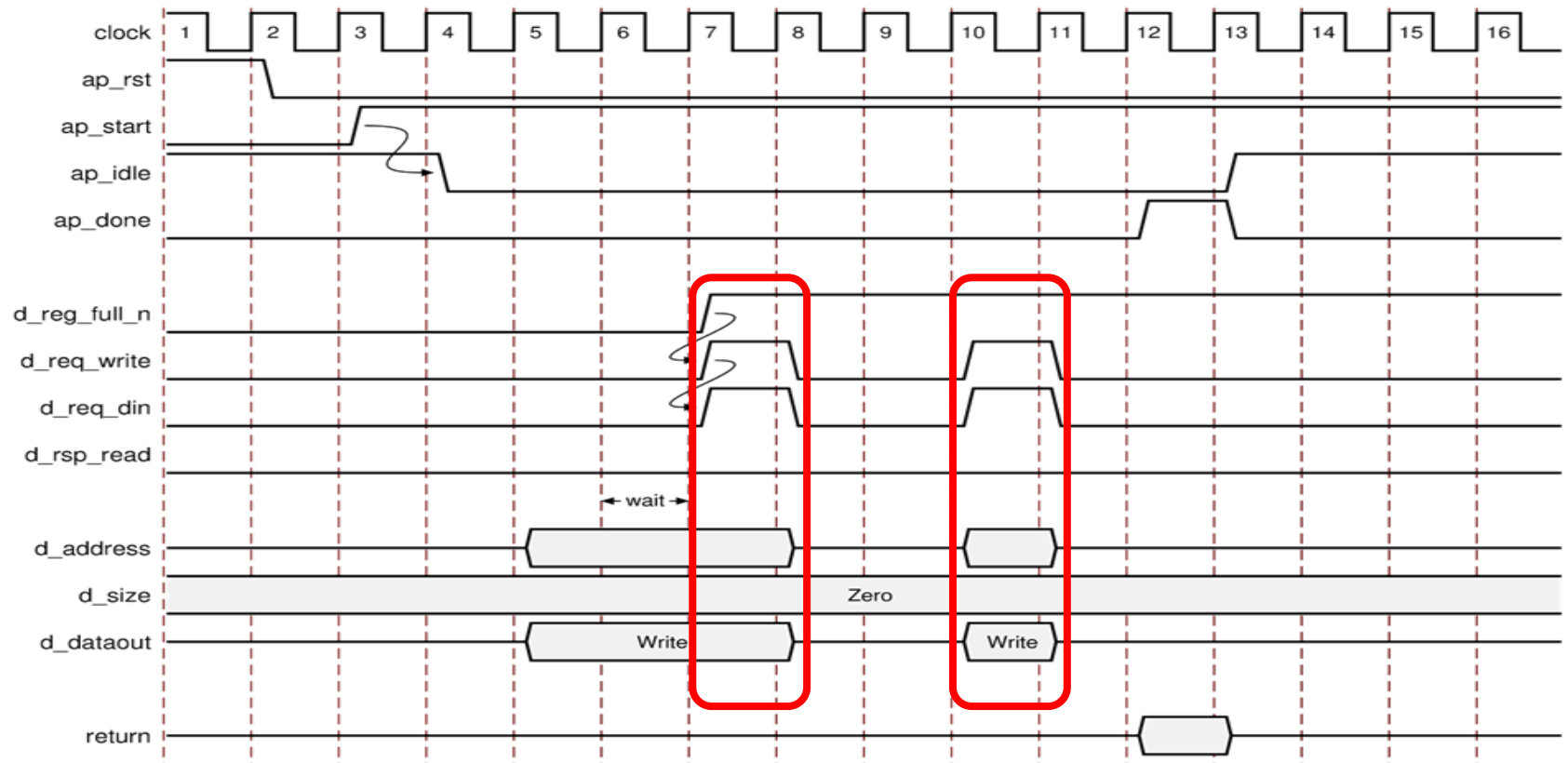
- Utóbbi vagy ciklusból, vagy memcpy()



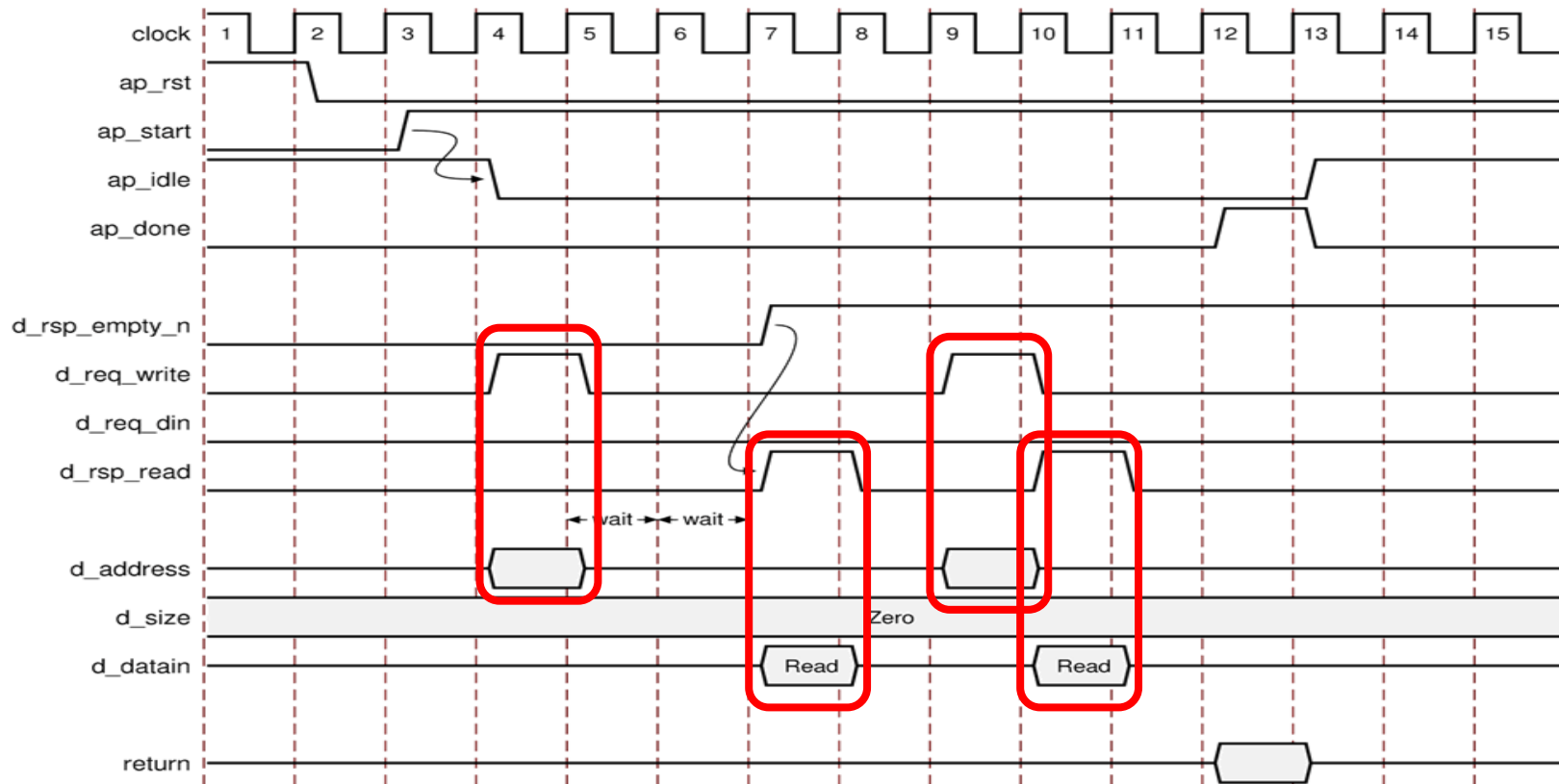
Bemeneti port	Funkció
d_datain	Bemeneti adat
d_req_full_n	A busz nem tud adatot fogadni. Írási igény esetén a HLS blokk várakozni fog. Aktív alacsony.
d_rsp_empty_n	A busz nem tud adtot kuldeni. Olvasási kérés estén a HLS blokk várakozni fog. Aktív alacsony.

Kimeneti port	Funkció
d_dataout	Kimeneti adat.
d_req_write	Átviteli kérés (írás vagy olvasás!)
d_req_din	Átvitel iránya. 0: read; 1: write.
d_rsp_read	Olvasási kérés esetén a tányleges adatolvasás kérése.
d_address	Cím (offset a báziscímhez képest).
d_size	Burst átvitel mérete.

Egyszeres írás

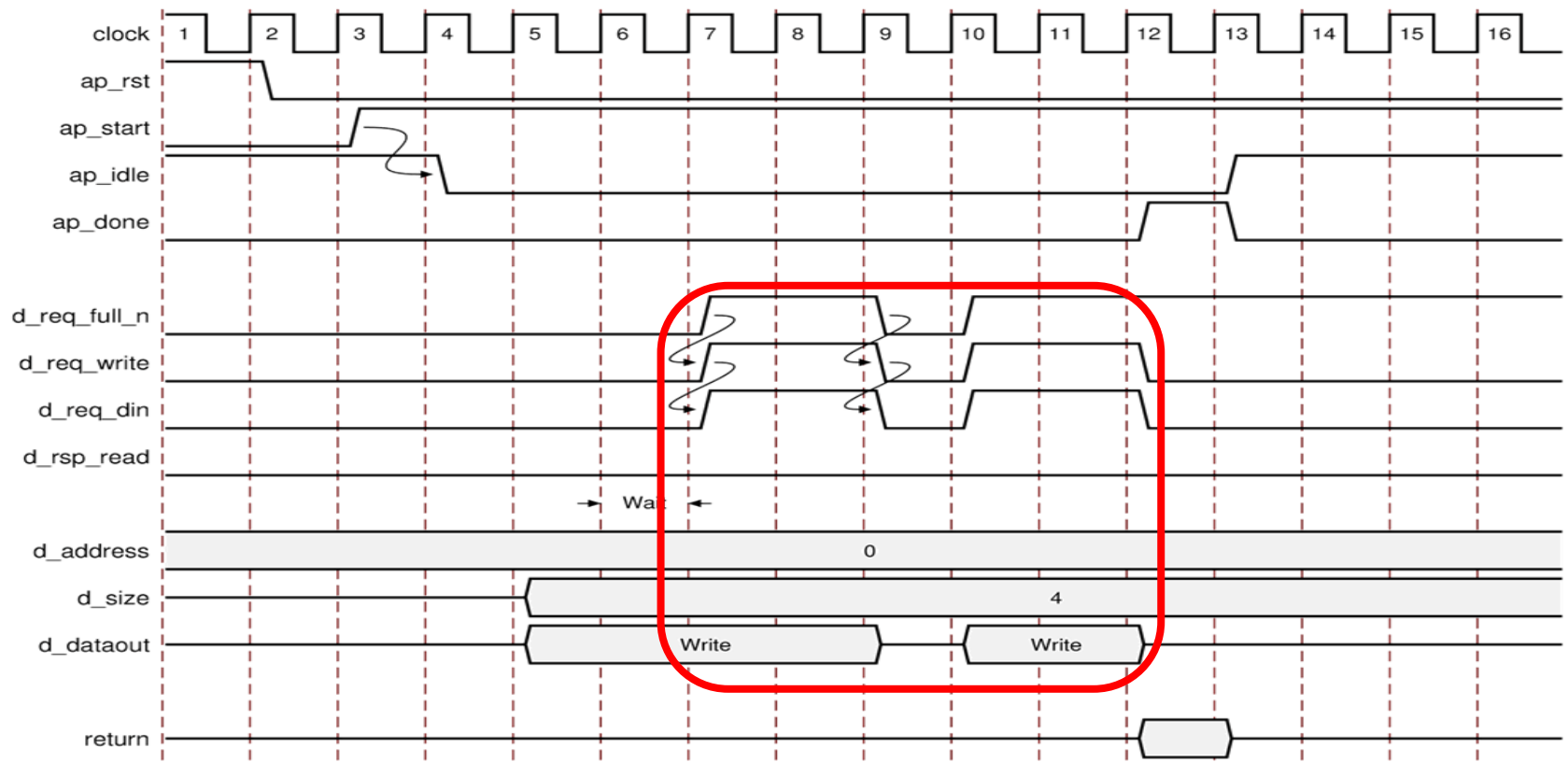


Egyszeres olvasás



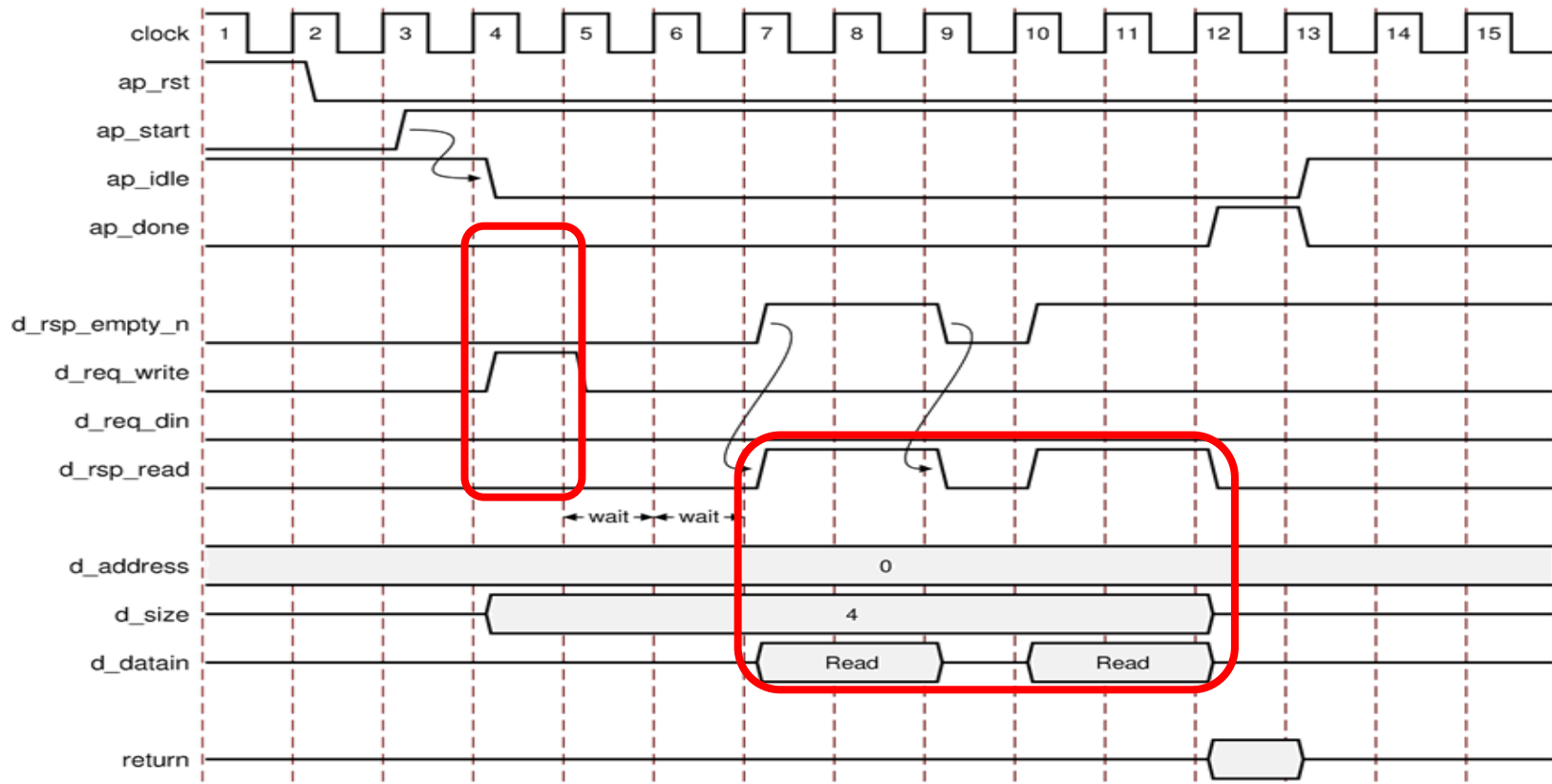
Burst írás

- Size=4 → 4 ciklusú burst
 - Req_write + req_din 4x aktív



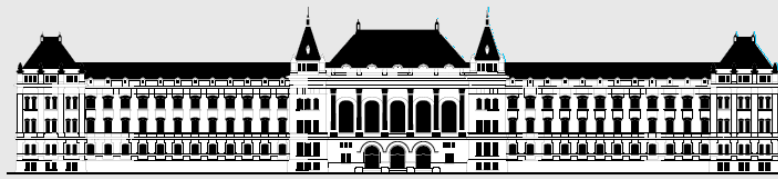
Burst olvasás

- 4 hosszú olvasási burst
 - 1x req_write + 4x rsp_read



AXI interfész

- **Regiszter hozzáféréshez AXI Lite slave**
 - Több port hozzárendelhető egy AXI Lite interfészhez → minden portból 1-1 regiszter lesz
- **Adatátvitelhez AXI master**
 - Maximális burst méret
 - Max. “outstanding read/write”
 - !!! Létrehoz annyi buffer-t, hogy ezeket tárolni tudja → ne tartsa fel a buszt
- **Példa később**

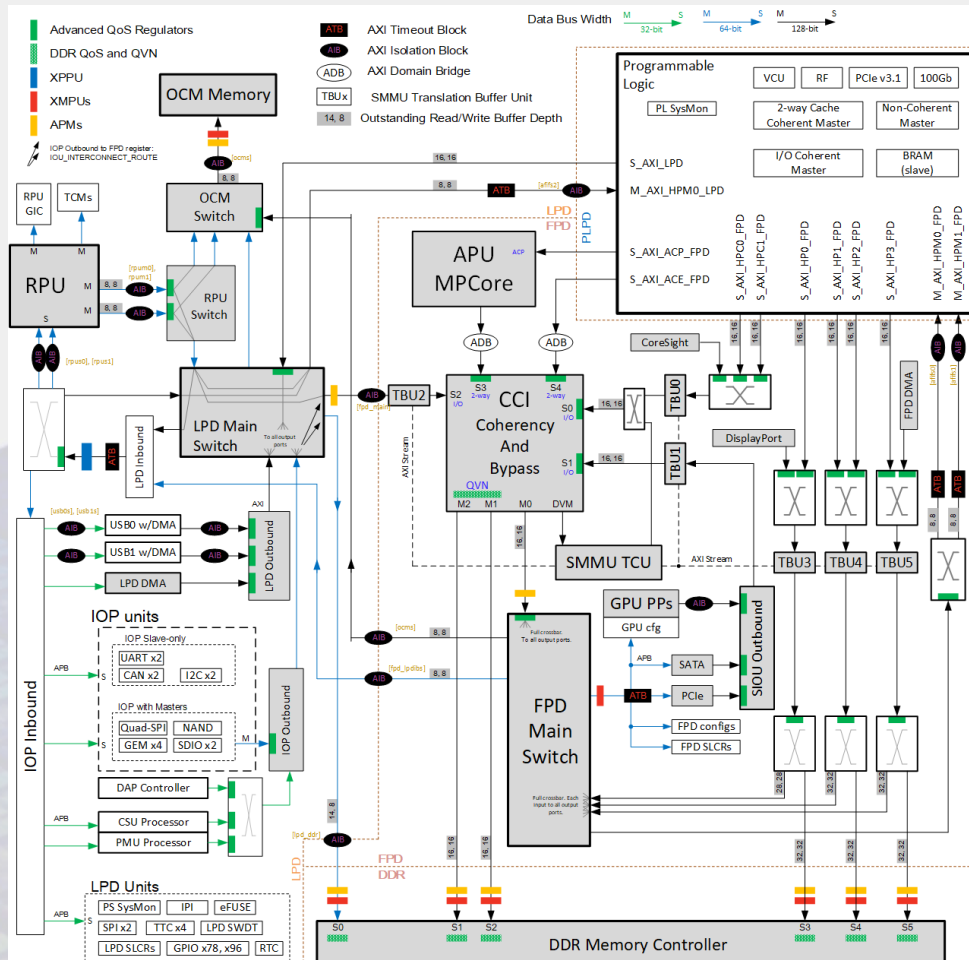


BUDAPESTI MŰSZAKI ÉS GAZDASÁGTUDOMÁNYI EGYETEM
VILLAMOSMÉRNÖKI ÉS INFORMATIKAI KAR
MESTERSÉGES INTELLIGENCIA ÉS RENDSZERTERVEZÉS TANSZÉK

Nagy méretű FIR (terem akusztika)

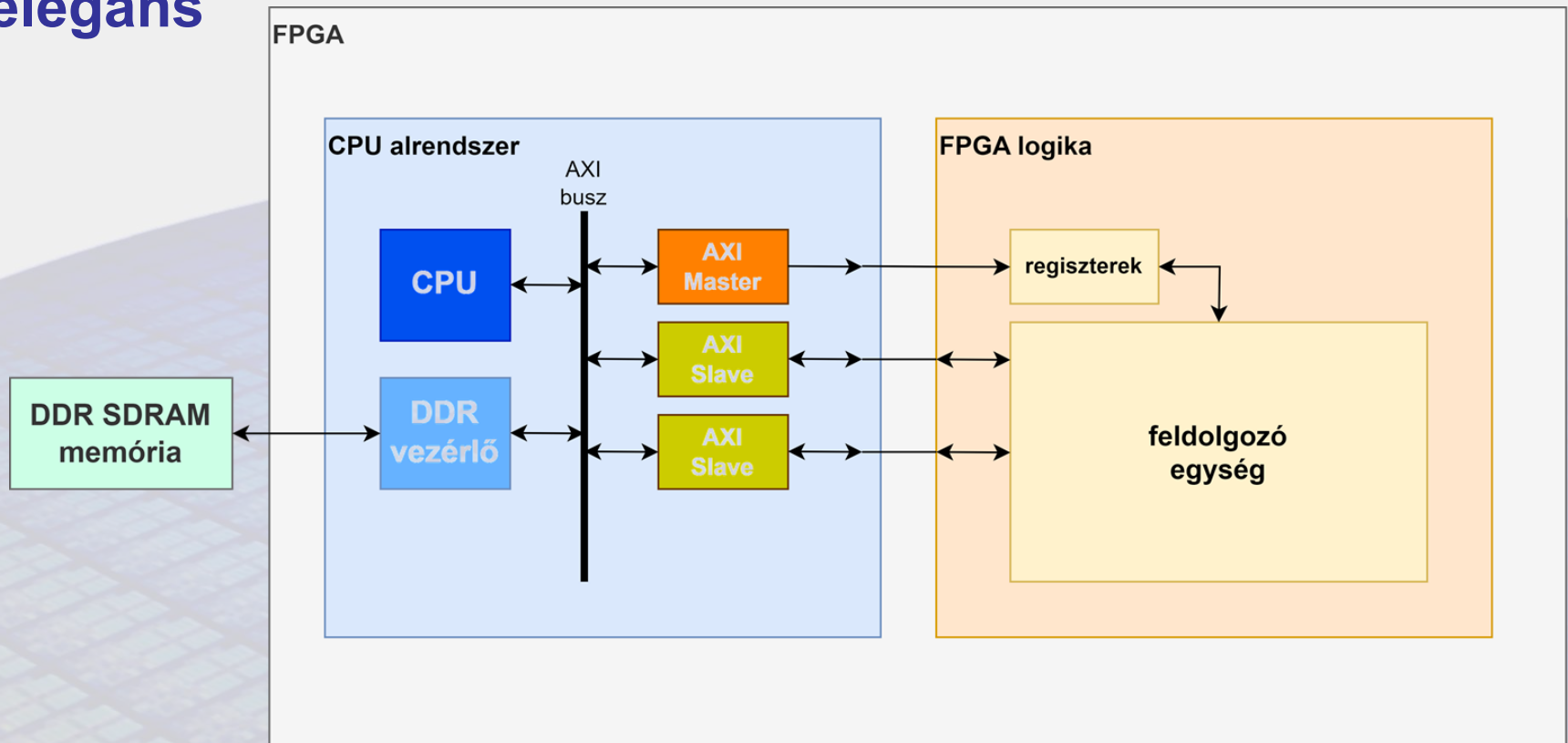
Platform

- Audio FIR szűrő AMD Kria K26 SOM-on
 - AMD UltraScale+ EV MPSoC



Audio FIR – blokkvázlat

- A CPU által vezérelhető hardveres gyorsító az extrém nagy méretű FIR szűrő megvalósítására
- Az együtttható elérne BRAM-ban (~4 Mbit), de nem elegáns



Audio FIR

- **A fejlesztő kezében van a kialakított hardver struktúra**
 - Nem kell meglévő (strukturális) tulajdonságokhoz alkalmazkodni
 - Azért az FPGA-ban levő elemek tulajdonságait, számát figyelembe kell venni
- **Limitek**
 - Az FPGA logika erőforrásai meghatározzák a megvalósítható számítási egységek számát
 - Sávzélesség
 - DDR SDRAM sávzélesség
 - FPGA – DDR RAM vezérlő közötti AXI interfészek sávzélessége

Adatformátum

- A GPU megvalósítás float adatokat használt, mert a GPU-k ezzel a leghatékonyabbak
- A valóságban a bemenetünk jó minőségű AD konvertert feltételezve 24 bites
- Ugyanez igaz az együtthatókra is, hiszen azokat is egy mikrofonnal vettük fel
- Az FPGA-ban 27x18 bites előjeles szorzó modul van
 - Két modul felhasználásával 27x35 bites számok szorozhatók
- A „szokásos” módon mind az együtthatókat, mind a mintákat -1...1 közé eső számoknak tekintjük → fix pontos számábrázolás
- Hardver tervezéskor mindig a minimálisan szükséges bitszámot használjuk → minimális erőforrás!

FIR – adattípusok

- **Minták: 24 bites, előjeles, 23 bit törtrész**

```
typedef ap_fixed<24, 1, AP_TRN, AP_WRAP> smp1_if_t;
```

- **Együttható: 32 bites, előjeles, 31 bit törtrész**

```
typedef ap_fixed<32, 1, AP_TRN, AP_WRAP> coeff_t;
```

- **Együttható és minta szorzata: 56 bites, előjeles, 54 tört**

```
typedef ap_fixed<56, 2, AP_TRN, AP_WRAP> mul_t;
```

- **Akkumulátor**

- Ha a legrosszabb esetre készülünk, akkor extrém méretű, mert rengeteg együtthatónk van
- 10 bittel nagyobb, mint a szorzat

```
typedef ap_fixed<66, 12, AP_TRN, AP_WRAP> accu_t;
```

- **Eredmény: szaturáljuk**

```
typedef ap_fixed<32, 1, AP_TRN, AP_SAT> res_t;
```

FIR – párhuzamosítás (1)

- CN: együtthatók száma
- Egy kimenet számítása (részleges) párhuzamosítással: N műveletvégző
 - CN/N lépésben

$$y_k = \underbrace{c_{CN} * s_k + c_{CN-1} * s_{k-1} + \dots + c_{CN-(N-1)} * s_{k-(N-1)}}_{N \text{ részszorzat}} +$$

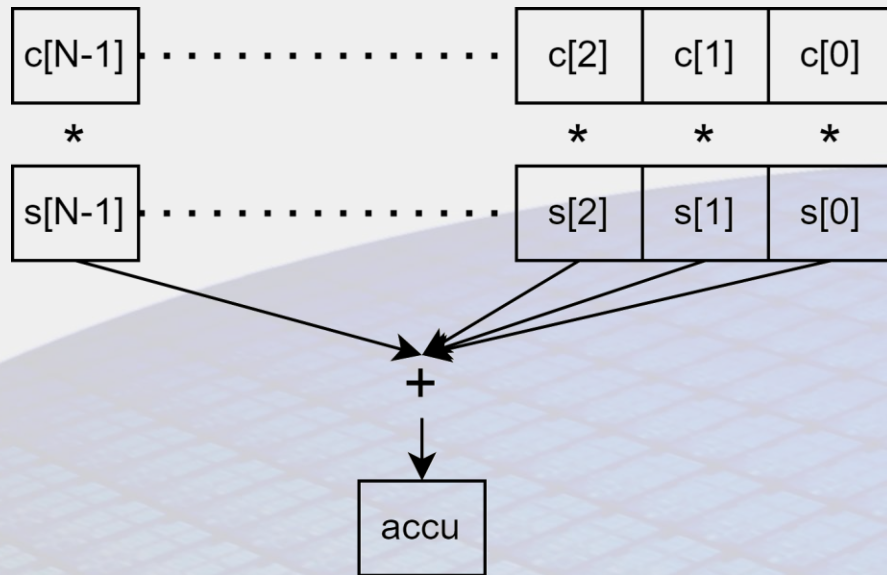
$$\underbrace{c_{CN-N} * s_{k-N} + c_{CN-(N+1)} * s_{k-(N+1)} + \dots + c_{CN-(2*N-1)} * s_{k-(2*N-1)}}_{N \text{ részszorzat}} +$$

.....

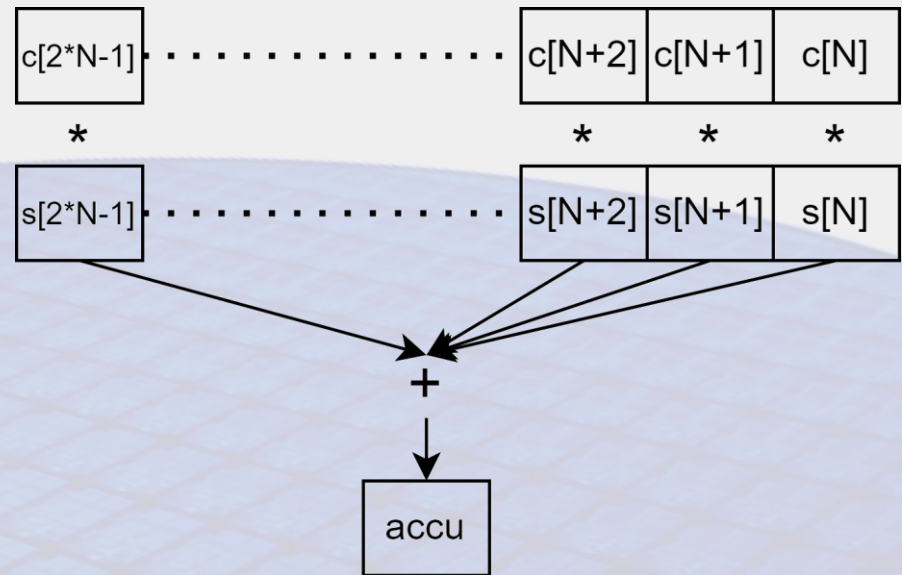
FIR – párhuzamosítás (2)

- Egy kimeneti minta számításának párhuzamosítása
 - N: MAC egységek száma (párhuzamosítás mértéke)

0. lépés



1. lépés



- Nagyon nagy a sávszélesség igény
 - N együttható beolvasás
 - N minta beolvasás

FIR – párhuzamosítás (3)

- Több kimenet számítása egyszerre N műveletvégzővel
 - CN lépésben N kimenet

The diagram illustrates the parallel calculation of N outputs of an FIR filter. It shows a sequence of equations for $y_k, y_{k-1}, y_{k-2}, \dots, y_{k-(N-2)}, y_{k-(N-1)}$. Each equation is a sum of products of coefficients c_i and samples s_j . The terms are grouped into three main sections: '0. lépés' (0th step), '1. lépés' (1st step), and '(CN-1). lépés' ((CN-1)th step). A red arrow points from the text 'N egymást követő kimenet' (N consecutive outputs) to the first output y_k .

$$\begin{array}{lcl} y_k & = & c_{CN} * s_k + c_{CN-1} * s_{k-1} + \dots + c_1 * s_{k-(CN-1)} \\ y_{k-1} & = & c_{CN} * s_{k-1} + c_{CN-1} * s_{k-2} + \dots + c_1 * s_{k-(CN-1)-1} \\ y_{k-2} & = & c_{CN} * s_{k-2} + c_{CN-1} * s_{k-3} + \dots + c_1 * s_{k-(CN-1)-2} \\ & \dots & \\ y_{k-(N-2)} & = & c_{CN} * s_{k-(N-2)} + c_{CN-1} * s_{k-(N-1)} + \dots + c_1 * s_{k-(CN-1)-(N-2)} \\ y_{k-(N-1)} & = & c_{CN} * s_{k-(N-1)} + c_{CN-1} * s_{k-(N)} + \dots + c_1 * s_{k-(CN-1)-(N-1)} \end{array}$$

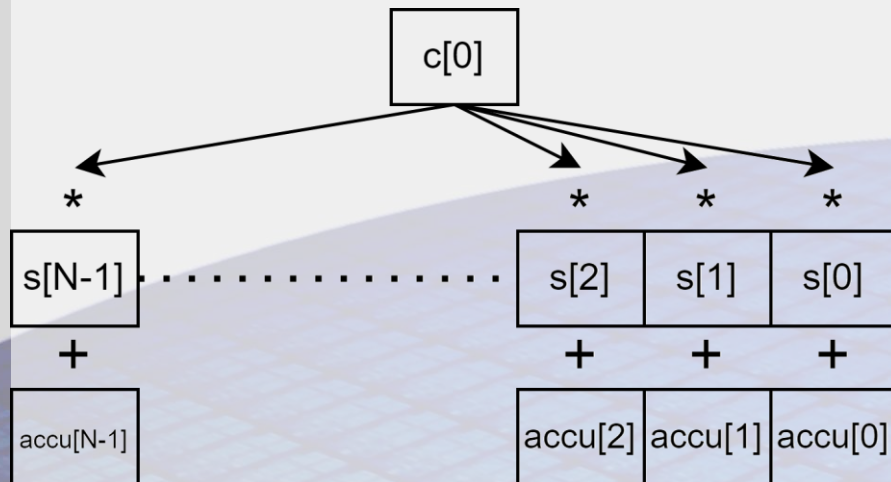
0. lépés 1. lépés (CN-1). lépés

N egymást
követő
kimenet

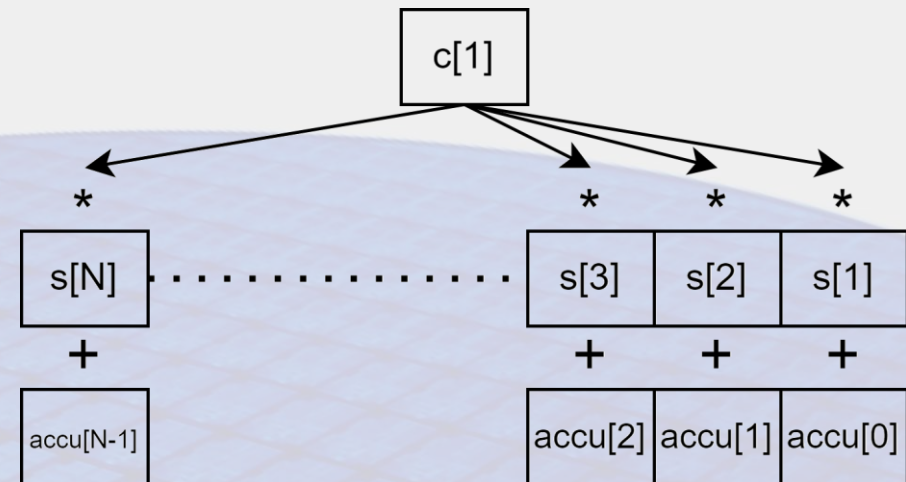
FIR – párhuzamosítás (4)

- Egyszerre több kimenet számítása
 - N MAC egység, N akkumulátor

0. lépés



1. lépés



- Elenyésző sáv szélesség igény $\rightarrow$ sokkal jobb
 - 1 új együttható
 - 1 új minta

MAC C függvény

- Az együtttható készlet sztereó, így kettesével olvassuk
- Az accu-k fele az egyik, fele a másik csatornát számolja
 - N: az egy csatornából számolt elemek száma $\rightarrow 2*N$ MAC egység
- **$2*N$ kimeneti mintát számol**
 - Inicializálás: N minta beolvasása (mintatár feltöltése)
 - Majd CN-szer (CN=együttthatók száma)
 - N-szer
 - MAC művlet: $\text{accu}[\text{ch}][i] = c[\text{ch}] * s[i]$
 - Új együttthatók (2 db) beolvasása
 - Új minta beolvasása (mintatár shift-elése)
 - $2*N$ darab eredmény kiírása

MAC C függvény

- Shift regiszter inicializálás
 - N darab kezdeti minta betöltése

```
void hls_mac(int res_block, int coeff_num,
             coeff_t *coeff, smpl_t *smpl, res_t *res)
{
    smpl_t smpl_shr[N];

    FOR_INIT: for (int i=0; i<N; i++) {
        smpl_t smpl_rd = *smpl;
        FOR_SH_INIT: for (int r=0; r<N; r++) {
            smpl_shr[r] = (r==(N-1)) ? smpl_rd : smpl_shr[r+1];
        }
    }
}
```


MAC init – pipeline

- Belső ciklusmag 256 órajel
- Külső: 256×256

LOOP NAME	MIN LATENCY(CYCLES)	MAX LATENCY(CYCLES)	LATENCY	ACHIEVED INITIATION INTERVAL	TARGET INITIATION INTERVAL	COUNT
- FOR_INIT_FOR_SH_INIT	65538	65538	4	1	1	65536

- Shift regisztert szeretnénk → a teljes belső ciklust egy órajel alatt kell végrehajtani
- → UNROLL pragma: kiteríti a ciklust

LOOP NAME	MIN LATENCY(CYCLES)	MAX LATENCY(CYCLES)	LATENCY	ACHIEVED INITIATION INTERVAL	TARGET INITIATION INTERVAL	COUNT
- FOR_INIT	513	513	2	-	-	256

- Jelen esetben hiába, mert a változó egy tömb

MAC init – partition

- Az egy órajeles shift-eléshez minden egyes tömb elemet írni és olvasni kell egyetlen órajelben → a tömb elemeket regiszterben (D FF) kell megvalósítani
- → ARRAY_PARTITION complete

LOOP NAME	MIN LATENCY(CYCLES)	MAX LATENCY(CYCLES)	LATENCY	ACHIEVED INITIATION INTERVAL	TARGET INITIATION INTERVAL	COUNT	PIPELINED
- FOR_INIT	513	513	2	-	-	256	no

- Még mindig nem elég jó, mert a ciklusmag végrehajtási ideje 2 órajel → teljes ciklus: 512+1

MAC init – pipeline + partition

- A külső ciklust pipeline-osítjuk → a belső ciklust egy órajel alatt végre kell hajtani (ha megvalósítható)
 - → automatikus UNROLL

LOOP NAME	MIN LATENCY(CYCLES)	MAX LATENCY(CYCLES)	LATENCY	ACHIEVED INITIATION INTERVAL	TARGET INITIATION INTERVAL	COUNT
- FOR_INIT	256	256	1	1	1	256

- Ez már rendben van, 256 adatot shift-elünk be 256 órajel alatt

MAC – init

- Végleges verzió

```
smp1_t smp1_shr[N];  
#pragma HLS ARRAY_PARTITION variable=smp1_shr dim=0 type=complete  
  
FOR_INIT: for (int i=0; i<N; i++) {  
#pragma HLS PIPELINE II=1  
    smp1_t smp1_rd = *smp1;  
    FOR_SH_INIT: for (int r=0; r<N; r++) {  
        smp1_shr[r] = (r==(N-1)) ? smp1_rd : smp1_shr[r+1];  
    }  
}
```

- Megj.: Vitis-ben sok pragma-nak nem kikapcsolt az alapállapota, illetve sokat automatikusan is használ
 - Csak nem mindig úgy, ahogy mi szeretnénk

MAC – ACCU

■ N*CN MAC → N eredmény

```
FOR_COEFF: for (int c=0; c<coeff_num; c++) {  
    coeff_r[0] = *coeff; coeff_r[1] = *coeff;  
    bool accu_rst = (c==0);  
    FOR_ACCU: for (int a=0; a<N; a++) {  
        mul[0][a] = coeff_r[0] * smpl_shr[a];  
        mul[1][a] = coeff_r[1] * smpl_shr[a];  
        accu[0][a] = (accu_rst==true) mul[0][a] : accu[0][a] + mul[0][a];  
        accu[1][a] = (accu_rst==true) mul[1][a] : accu[1][a] + mul[1][a];  
    }  
    smpl_t smpl_rd = *smpl;  
    FOR_SH_SMPL: for (int r=0; r<N; r++) {  
        smpl_shr[r] = (r==(N-1)) ? smpl_rd : smpl_shr[r+1];  
    }  
}  
}
```

MAC C függvény

- Minden órajelben szeretnénk újabb N darab MAC műveletet elkezdni
 - FOR_ACCU: UNROLL
 - FOR_SH_SMPL: UNROLL
 - Accu: ARRAY_PARTITON complete
 - → Nagy a FOR_COEFF ciklusmagjának késleltetése → nem jó
- → **FOR_COEFF: pipeline**

LOOP NAME	MIN LATENCY(CYCLES)	MAX LATENCY(CYCLES)	LATENCY	ACHIEVED INITIATION INTERVAL	TARGET INITIATION INTERVAL	COUNT	PIPELINED
- FOR_COEFF	109148	109148	7	1	1	109143	yes

- Iteráció szám 109143, mert a használt együttható készlet ennyi elemet tartalmaz
- Kb ennyi a teljes ciklus végrehajtási ideje is

Eredmény kiírása

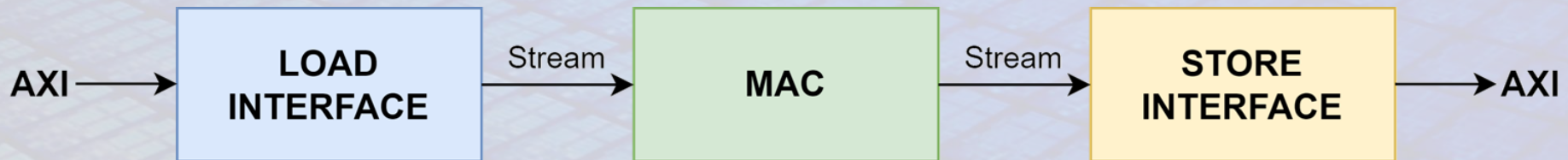
- Az $2*N$ accu értéket kiírjuk

```
.....  
FOR_ST_RES: for (int r=0; r<N; r++) {  
    *(res) = accu[0][r];  
    *(res) = accu[1][r];  
}
```

- Ez $2*N$ órajelet vesz igénybe, mert egyetlen interfészünk van

Top-level függvény

- A fentiekben N darab eredményt számoltunk ki csatornánként
- A teljes bemeneti adat feldolgozásához ezt sokszor (bemeneti minták száma / N) meg kell tennünk
 - → A MAC függvényt többször hajtjuk végre
- A top-level függvény argumentumaiból AXI busz interfészt szeretnénk létrehozni
 - Ezek a DDR memóriából olvasnak/oda írnak
 - A memóriát címezni kell → ezt a C kódban kell megoldani
 - Szétválasztjuk az AXI interfészeket és a számítást
 - → Ezek között a kapcsolat: HLS Stream (FIFO)



HLS Stream

- **Deklaráció**

```
#include <hls_stream.h>

hls::stream<elem típus, mélység> myStream();
```

- **Alapbeállítás szerint a mérete 2 elem**

- STREAM pragma-val állítható
- Lehet FIFO vagy ping-pong buffer

- **Blokkoló olvasás**

```
adat = myStream.read();
```

- **Blokkoló írás**

```
myStream.write(adat);
```

Load/Store

- **A LOAD blokk annyi adatot olvas, amennyire a MAC-nek egy „híváshoz” szüksége van**
 - $2 \cdot CN$ együttható
 - $N + CN$ minta
- **A STORE annyit ír, amennyi kimenetet a MAC generál egy hívásra**
 - $2 \cdot N$
- **A 3 függvény NEM hajtható végre egymás után, mert a bemeneti adatok buffereléséhez nincs elég on-chip memória**
 - És lassú is lenne
- **Párhuzamosan működnek → DATAFLOW**

Load/Store – Dataflow

- A rendszer akkor hatékony, ha minden egyes blokk kb. ugyanannyi idő alatt végez
 - Ha valamelyik lassabb, akkor feltartja a többi
 - Gyorsabb felesleges
- A MAC 2 együtthatót (bal/jobbs) olvas órajelenként
 - → A Stream-nek ezt ki kell tudnia szolgálni
 - Típusa 2 elemű vektor
- **hls::vector: max 4096 bites**

```
typedef hls::vector<elem_típus, elem_szám> típus_név;
```

- Esetünkben

```
typedef hls::vector<coeff_t, 2> coeff2vec_t;
```

Load

▪ Együttható és minta betöltés

```
void hls_load(int res_block, int coeff_num,
             coeff2vec_t *coeff_if, hls::stream<coeff2vec_t> &coeffStream,
             smpl_if_t smpl_if, hls::stream<smpl_t> &smplStream)
{
    FOR_LD_INIT: for (int ld=0; ld<N; ld++) {
        smpl_if_t smpl_rd = smpl_if[res_block*N + ld];
        smplStream.write((smpl_t)(smpl_rd));
    }

    FOR_LD: for (int ld=0; ld<coeff_num; ld++) {
#pragma HLS LOOP_TRIPCOUNT max=109143 avg=109143 min=109143
        coeff2vec_t coeff_rd = coeff_if[ld];
        coeffStream.write(coeff_rd);

        smpl_if_t smpl_rd = smpl_if[res_block*N + N + ld];
        smplStream.write((smpl_t)(smpl_rd));
    }
}
```


Store

■ Eredmény kiírása

```
void hls_store(int res_block, hls::stream<res2vec_t, N>
&resStream, res2vec_t *res_if)
{
    FOR_RES: for (int res=0; res<N; res++) {
        *(res_if + res_block*N + res) = resStream.read();
    }
}
```

- A kimeneti adatformátumunk is 2 elemű vektor
- Nem interfész sebesség miatt, az sokkal gyorsabb a kelletténél

Top-level

■ Legfelső szintű függvény

```
void hls_test(int res_blocks, int coeff_num, coeff2vec_t *coeff_if,
             smpl_if_t *smpl_if, res2vec_t *res_if)
{
    #pragma HLS INTERFACE mode=m_axi port=coeff_if bundle=axi_mm_0
    #pragma HLS INTERFACE mode=m_axi port=smpl_if bundle=axi_mm_1
    #pragma HLS INTERFACE mode=m_axi port=res_if bundle=axi_mm_0

    for (int res_block=0; res_block<res_blocks; res_block++){
        #pragma HLS LOOP_TRIPCOUNT max=38795 avg=38795 min=38795
        #pragma HLS DATAFLOW
        static hls::stream<coeff2vec_t, N> coeffStream("coeff_stream0");
        static hls::stream<smpl_t, N> smplStream("smpl_stream");
        static hls::stream<res2vec_t, N> resStream("res_stream");

        hls_load(res_block, coeff_num, coeff_if, coeffStream, smpl_if,
smplStream);
        hls_mac(res_block, coeff_num, coeffStream, smplStream, resStream);
        hls_store(res_block, resStream, res_if);
    }
}
```

Eredmény

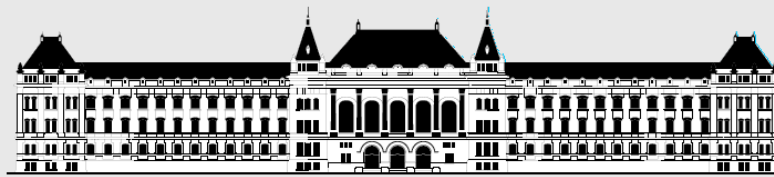
- **Elvi feldolgozási idő, N=256**
 - Impulzus válasz hossza (per csatorna): 109143
 - Bemeneti mintaszám: 9822291
 - Számolt kimenet: 9931433 minta/csatorna
 - 2 csatorna
 - $(2 * 9931433 * 109143) / (2 * 256) = 4\,234\,165\,593$ órajel

Eredmény

■ C szintézis eredménye

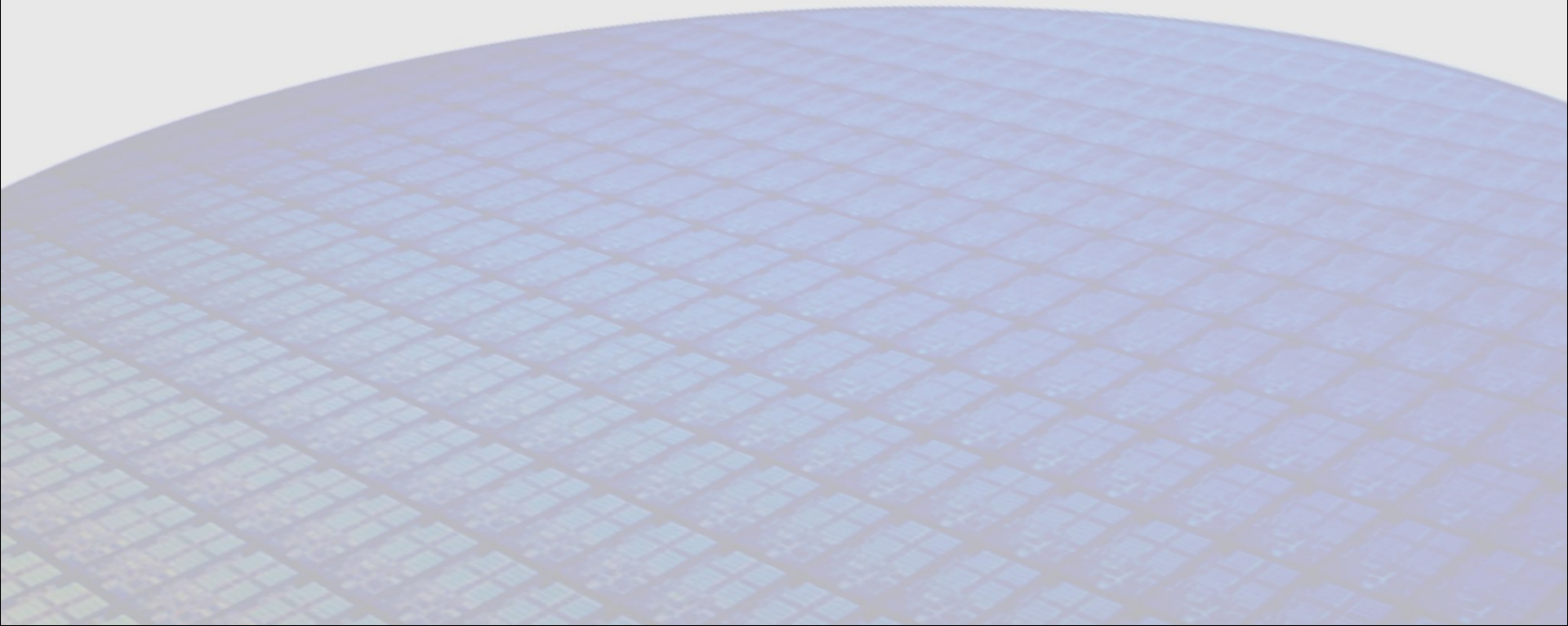
▼ ● fir_audio_hls (1)	00		4,254,686,445		no	function	14	1,024	162,358	104,653	0
▼ ● VITIS_LOOP_113_1 (1)	00	4,254,686,521	109,671	38,795	dataflow	loop					
▼ ● dataflow_in_loop_VITIS_LOOP_113_1_1 (4)	E5		109,671		dataflow	function	10	1,024	161,144	103,083	0
● entry_proc	0.0		0		no	function	0	0	3	38	0
> ● hls_load2 (2)	E5		109,478		no	function	0	0	320	916	0
▼ ● hls_mac3 (3)	E5		109,670		no	function	0	1,024	159,701	101,484	0
▼ ● hls_mac3_Pipeline_FOR_COEFF (1)	E5		109,144		loop auto-rew	function	0	1,024	153,349	98,295	0
● FOR_COEFF	E5	5	1	109,143	yes	loop					
> ● hls_mac3_Pipeline_FOR_ST_RES (1)	00		257		loop auto-rew	function	0	0	152	3,034	0
> ● hls_mac3_Pipeline_FOR_INIT (1)	00		257		loop auto-rew	function	0	0	6,158	84	0
> ● hls_store4 (1)	00		256		loop auto-rew	function	0	0	344	243	0

- Elvi számítási idő: 4 234 165 593 órajel
- C szintézis eredmény: 4 254 958 087 órajel (99,5%)
 - → 8.5 sec @ 500 MHz
 - → 17 sec @ 250 MHz



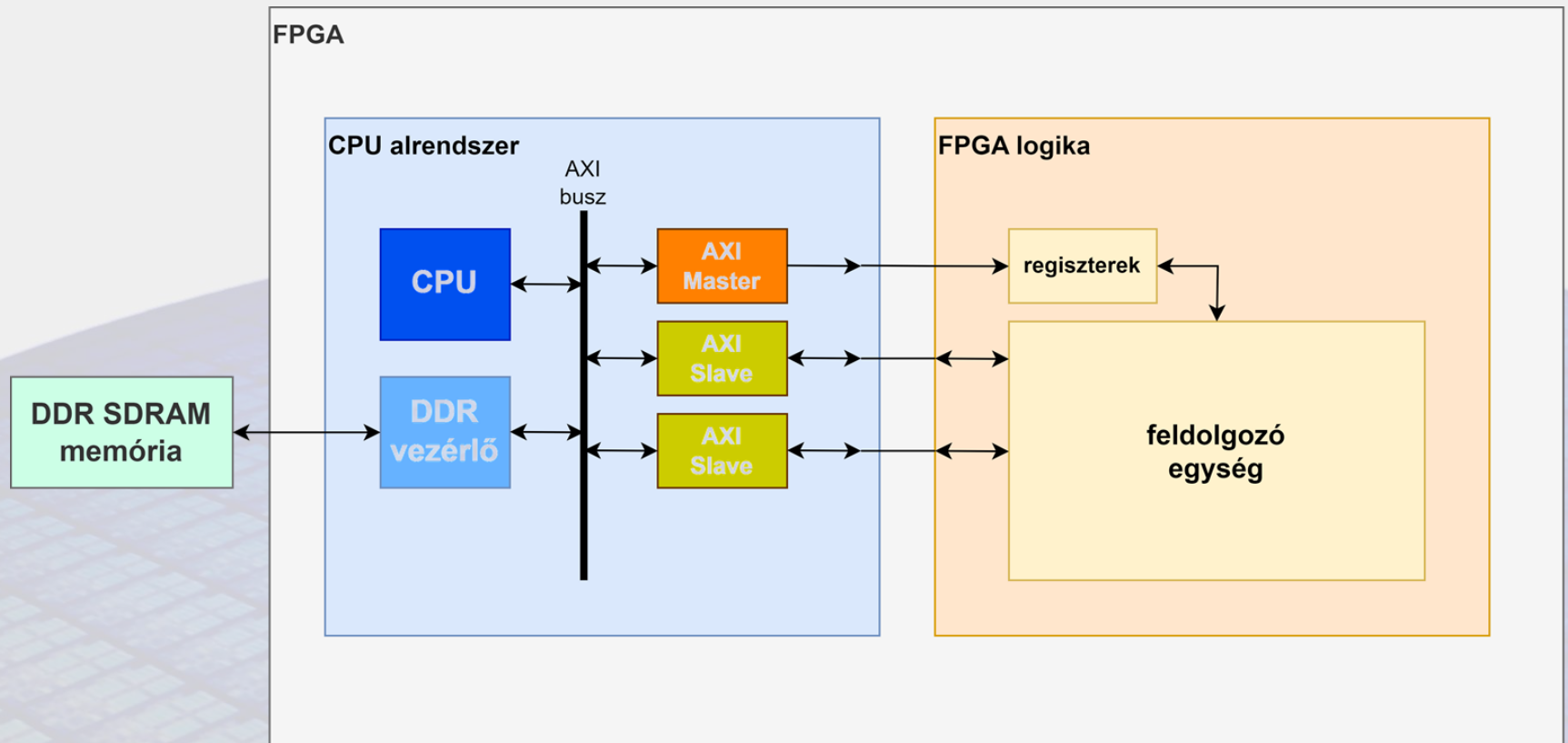
BUDAPESTI MŰSZAKI ÉS GAZDASÁGTUDOMÁNYI EGYETEM
VILLAMOSMÉRNÖKI ÉS INFORMATIKAI KAR
MESTERSÉGES INTELLIGENCIA ÉS RENDSZERTERVEZÉS TANSZÉK

Mátrix szorzás



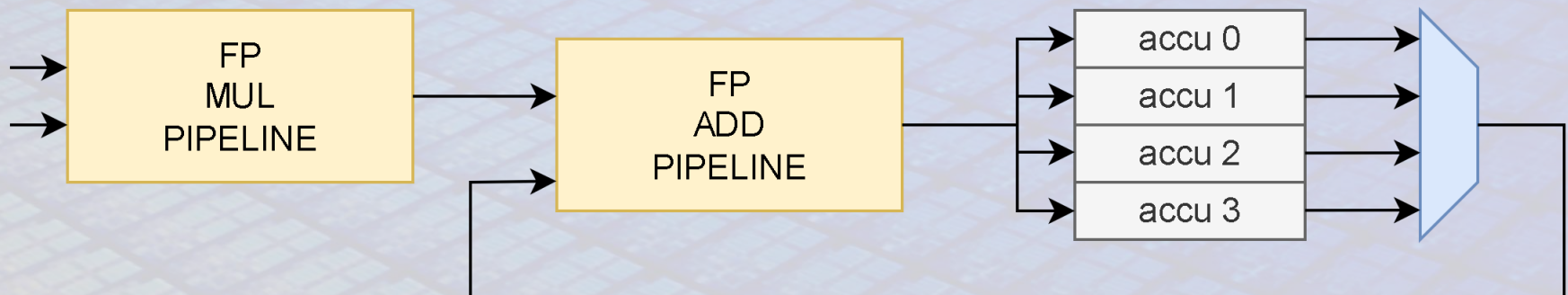
Hardver blokkvázlat

- Az alapvető struktúra ugyanaz, mint a FIR szűrő esetében



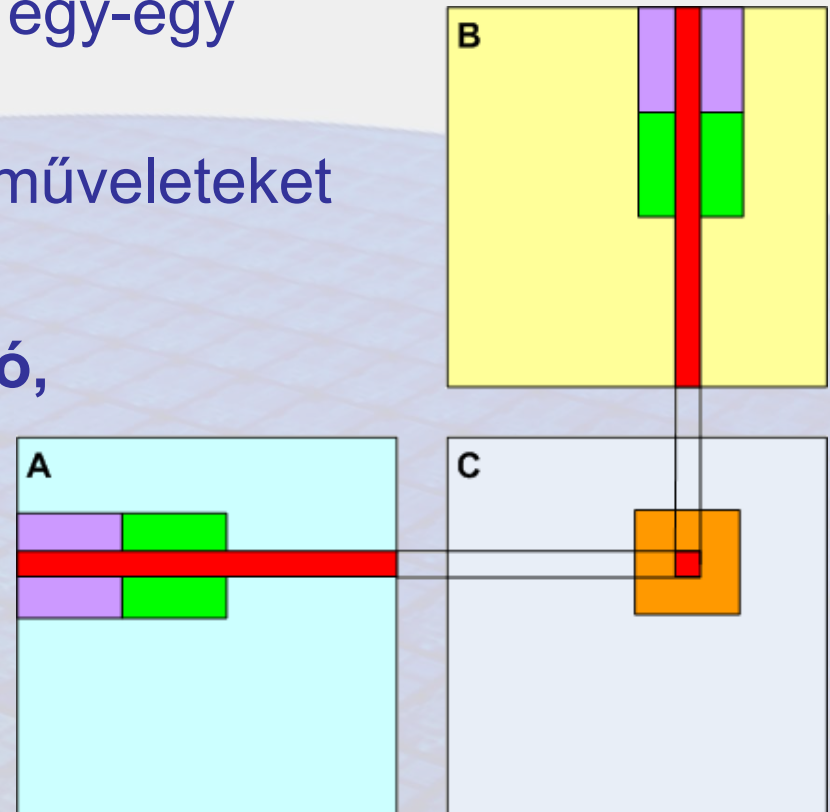
Float MAC

- **A mátrix szorzás alaplővelete a MAC**
 - Visszacsatolt összeadót tartalmaz
- **A lebegőpontos összeadás bonyolult logika, értelmes működési frekvencia eléréséhez egy viszonylag hosszú pipeline-nal rendelkező műveletvégző**
- **Hatékony kihasználásához időosztásban egymástól független értékeket kell számolni**
 - Minimum annyit, amennyi a pipeline latency



Struktúra (1)

- Alapvetően ugyanazt csináljuk, mint GPU-n
- Blokkosan haladunk végig a kimeneti mátrixon
 - Egy kimeneti blokk számításához az alábbiakat ismételjük
 - Betöltjük az A és B matrix egy-egy részét
 - Elvégezzük a lehetséges műveleteket
 - Kiírjuk az eredményeket
- Hardver realizációról lévén szó, a töltés – számítás műveleteket párhuzamosan hajtjuk végre



Struktúra (2)

- A párhuzamosítás mértékét elsősorban az adott FPGA-ban megvalósítható MAC egységek száma szabja meg
- Az A és B rész-mátrixok betöltése nem tarthat tovább, mint a betöltött adatok feldolgozása → ellenkező esetben memória sávszélesség limit lesz
- N MAC egység és $N \times N$ méretű blokk használata esetén
 - $N \times N \times N$ MAC művelet → $N \times N$ órajel (< 500 MHz)
 - $2 \times N \times N$ adat töltés → $8 \times N \times N$ byte
 - CPU – PL AXI busz sávszélesség
 - Max 333 MHz
 - Max 128 bit → 16 byte → egy busz elegendő
 - Memória sávszélesség
 - 64-bit @ 2400 Mb/s → egy FPGA órajel alatt kiolvasható 8 byte
 - Burst hozzáférés kell!
 - DDR memória olvasási késleltetése nagy!

Top-level

```
#define N 1024
#define BLOCK 128

float mem_a[BLOCK][BLOCK];
float mem_b[BLOCK][BLOCK];
float accu[BLOCK][BLOCK];
float out_ram[BLOCK][BLOCK];

void mmul_hw(volatile float *a_base, volatile float *b_base, volatile float *res_base)
{
#pragma HLS INTERFACE mode=m_axi depth=256 port=a_base bundle=gmem_rd num_read_outstanding=4
#pragma HLS INTERFACE mode=m_axi depth=256 port=b_base bundle=gmem_rd num_read_outstanding=4
#pragma HLS INTERFACE mode=m_axi depth=256 port=res_base bundle=gmem_res num_write_outstanding=4
#pragma HLS BIND_STORAGE variable=mem_b type=RAM_2P impl=LUTRAM
#pragma HLS BIND_STORAGE variable=accu type=RAM_2P impl=LUTRAM
#pragma HLS BIND_STORAGE variable=out_ram type=RAM_2P impl=LUTRAM
    STEP_Y: for (int out_y=0; out_y<N; out_y=out_y+BLOCK) {
        STEP_X: for (int out_x=0; out_x<(N/BLOCK); out_x++) {
#pragma HLS DATAFLOW
            BLOCK_STEP: for (int step=0; step<(N/BLOCK); step++) {
#pragma HLS DATAFLOW
                // load A & B sub-matrices
                load_din(a_base, b_base, out_y, out_x, step);
                // compute
                comp_mmul(step);
            }
            // store
            store_res(out_y, out_x*BLOCK, res_base);
        }
    }
}
```

Adat betöltés

```
void load_din(volatile float *a_base, volatile float *b_base, int out_y, int out_x, int step)
{
    int a_addr = out_y*N + step*BLOCK;
    int b_addr = out_x*BLOCK + (step*BLOCK)*N;

    LOAD_BLOCK_Y:for (int load=0; load<BLOCK; load++) {
#pragma HLS PIPELINE

        memcpy((float*)&mem_a[load][0]),
               (float*)(a_base + a_addr),
               sizeof(float)*BLOCK);

        memcpy((float*)&mem_b[load][0]),
               (float*)(b_base + b_addr),
               sizeof(float)*BLOCK);
        a_addr= a_addr + N;
        b_addr = b_addr + N;
    }
}
```

- Burst méret: megegyezik a blokk mérettel
- A PIPELINE direktíva alapértelmezetten ll=1, itt ez biztosan nem fog teljesülni → ez csak warning lesz

MAC

- Egy $BLOCK \times BLOCK$ méretű kimeneti terület minden elemére $BLOCK$ számú részszorzat
 - A 2D blokkon való végig lépkedés kód szinten egy ciklusban

```
void comp_mmul(int step)
{
    float a_data[BLOCK];
    float b_data[BLOCK];
    int init_int;
    init_int = (step==0);
    int last_val = 0;
    BSTEP_MACY: for (int i=0; i<BLOCK*BLOCK; i++) {
#pragma HLS PIPELINE II=1
        ap_uint<5> bstep = i/BLOCK;
        ap_uint<5> y = i%BLOCK;
        MAC_X: for(int x=0; x<BLOCK; x++) {
            float mul_res = mem_a[y][bstep] * mem_b[bstep][x];
            float accu_rd = accu[y][x];
            if (init_int)      accu[y][x] = mul_res;
            else if (last_val) out_ram[y][x] = accu_rd + mul_res;
            else               accu[y][x] = accu_rd + mul_res;
        }
        init_int = 0;
        last_val = (bstep == (BLOCK - 2)) & (step == ((N / BLOCK)-1));
    }
}
```


Eredmények kiírása

- **BLOCK** számú kimeneti sor
 - **BLOCK** hosszú memcpy

```
void store_res(int out_y, int out_x, volatile float *res_base)
{
    int addr = out_y*N + out_x;
    WR_LINE: for (int line=0; line<BLOCK; line++) {
#pragma HLS PIPELINE
        memcpy((float*)(res_base+addr), (float*)&out_ram[line][0], BLOCK*sizeof(float));
        addr = addr + N;
    }
}
```

C szintézis eredmény (1)

- **1024x1024 méretű matrix, BLOCK=128**
 - 128 MAC, 128x128 méretű rész-matrix
 - Megj.: a C tömbök particionálását a Vitis HLS-re hagytuk, ezt meg is tette automatikusan
- **Elvben minimum $1024*1024*1024/128=8\ 388\ 608$ órajel**

MODULES & LOOPS	IS	VH TV	DI	SLACK	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
▼ mmul_hw (1)				-0.50	2.938E7		8,388,608		no	function	68	640	132,273	91,234	0
▼ C_STEP_Y (1)					2.938E7	8,396,902	1,048,576	8	dataflow	loop					
▼ dataflow_parent_loop_proc (1)				-0.50	3.698E6		1,048,576		no	function	60	640	130,621	89,252	0
▼ C_STEP_X (1)					3.698E6	1,056,868	131,072	8	dataflow	perfect loop					
▼ dataflow_in_loop_STEP_X_1 (2)				-0.50	4.880E5		131,072		dataflow	function	60	640	130,613	89,210	0
▼ dataflow_in_loop_STEP_X_1_Loop_BLOCK_STEP_proc54 (1)				-0.50	4.880E5		131,072		no	function	60	640	130,406	88,270	0
▼ C_BLOCK_STEP (1)					4.880E5	139,361	16,384	8	dataflow	loop					
▼ dataflow_in_loop_BLOCK_STEP_176 (2)				-0.50	8.633E4		16,384		dataflow	function	60	640	130,398	88,228	0
▼ load_din387 (1)					2.480E4		8,192		loop auto-rew	function	0	0	4,221	1,950	0
C_LOAD_BLOCK_Y			Re		2.480E4	138	64	128	yes	loop					
▼ comp_mmul49 (1)					4.921E4		16,384		loop auto-rew	function	0	640	109,692	69,799	0
C_BSTEP_MACY					4.920E4	19	1	16,384	yes	loop					
▼ store_res65 (1)					4.937E4		16,384		loop auto-rew	function	0	0	205	906	0
C_WR_LINE			Mr	1	4.936E4	198	128	128	yes	loop					

■ Örülünk 😊

C szintézis eredmény (2)

■ Erőforrás

- DSP: 640
 - FMUL: 3, FADD: 2
- BRAM: 68
 - a_mem: 58; 128x128 float, 2 bank (dual buffer)
 - Maradék a busz interfész
- LUTRAM
 - b_mem: 128 db 128x32, 2 bank
 - accu: 128 db 128x32, 1 bank

■ Elérhető órajel frekvencia

- C szintézis min. órajel periódus: 3,5 ns
- Implementáció után:

C szintézis eredmény (3)

■ mem_b is BlockRam-ban

MODULES & LOOPS	IS	VH TY	DI	SLACK	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
▼ mmul_hw (1)				-0.50	2.938E7		8,388,608		no	function	324	640	115,889	74,850	0
▼  STEP_Y (1)					2.938E7	8,396,902	1,048,576	8	dataflow	loop					
▼ mmul_hw (1) dataflow_parent_loop_proc (1)				-0.50	3.698E6		1,048,576		no	function	316	640	114,237	72,868	0
▼  STEP_X (1)					3.698E6	1,056,868	131,072	8	dataflow	perfect loop					
▼  dataflow_in_loop_STEP_X_1 (2)				-0.50	4.880E5		131,072		dataflow	function	316	640	114,229	72,826	0
▼ mmul_hw (1) dataflow_in_loop_STEP_X_1_Loop_BLOCK_STEP_proc54 (1)				-0.50	4.880E5		131,072		no	function	316	640	114,022	71,886	0
▼  BLOCK_STEP (1)					4.880E5	139,361	16,384	8	dataflow	loop					
▼  dataflow_in_loop_BLOCK_STEP_176 (2)				-0.50	8.633E4		16,384		dataflow	function	316	640	114,014	71,844	0
▼ mmul_hw (1) load_din387 (1)					2.480E4		8,192		loop auto-rew	function	0	0	4,221	1,950	0
 LOAD_BLOCK_Y		ii	Re		2.480E4	138	64	128	yes	loop					
> mmul_hw (1) comp_mmul49 (1)					4.921E4		16,384		loop auto-rew	function	0	640	109,692	69,799	0
▼ mmul_hw (1) store_res65 (1)					4.937E4		16,384		loop auto-rew	function	0	0	205	906	0
 WR_LINE		ii	Mt 1		4.936E4	198	128	128	yes	loop					